In [1]:
```
%pip install scikit-learn
%pip install seaborn
```

```
Requirement already satisfied: scikit-learn in C:\Users\wahee\AppData\Local\Python\p
ythoncore-3.14-64\Lib\site-packages (1.9.1)
Requirement already satisfied: numpy>=1.24.1 in C:\Users\wahee\AppData\Local\Python
\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (2.5.3)
Requirement already satisfied: scipy>=1.10.0 in C:\Users\wahee\AppData\Local\Python
\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (1.18.1)
Requirement already satisfied: joblib>=1.4.0 in C:\Users\wahee\AppData\Local\Python
\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: narwhals>=2.0.1 in C:\Users\wahee\AppData\Local\Pytho
n\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (2.27.1)
Requirement already satisfied: threadpoolctl>=3.5.0 in C:\Users\wahee\AppData\Local
\Python\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in C:\Users\wahee\AppData\Local\Pyth
on\pythoncore-3.14-64\Lib\site-packages (from joblib>=1.4.0->scikit-learn) (3.1.2)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: seaborn in C:\Users\wahee\AppData\Local\Python\python
core-3.14-64\Lib\site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in C:\Users\wahee\AppData\Local
\Python\pythoncore-3.14-64\Lib\site-packages (from seaborn) (2.5.3)
Requirement already satisfied: pandas>=1.2 in C:\Users\wahee\AppData\Local\Python\py
thoncore-3.14-64\Lib\site-packages (from seaborn) (3.0.6)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in C:\Users\wahee\AppData\Loc
al\Python\pythoncore-3.14-64\Lib\site-packages (from seaborn) (3.11.2)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\wahee\AppData\Local\Pyth
on\pythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.
4.0)
Requirement already satisfied: cycler>=0.10 in C:\Users\wahee\AppData\Local\Python\p
ythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in C:\Users\wahee\AppData\Local\Pyt
hon\pythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.
66.1)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\wahee\AppData\Local\Pyt
hon\pythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.
5.1)
Requirement already satisfied: packaging>=20.0 in C:\Users\wahee\AppData\Local\Pytho
n\pythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.
3)
Requirement already satisfied: pillow>=9 in C:\Users\wahee\AppData\Local\Python\pyth
oncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.3.0)
Requirement already satisfied: pyparsing>=3 in C:\Users\wahee\AppData\Local\Python\p
ythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.3)
Requirement already satisfied: python-dateutil>=2.7 in C:\Users\wahee\AppData\Local
\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(2.9.0.post0)
Requirement already satisfied: tzdata in C:\Users\wahee\AppData\Local\Python\pythonc
ore-3.14-64\Lib\site-packages (from pandas>=1.2->seaborn) (2026.4)
Requirement already satisfied: six>=1.5 in C:\Users\wahee\AppData\Local\Python\pytho
ncore-3.14-64\Lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4-
>seaborn) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers,datasets
from sklearn.model_selection import train_test_split
import random
import pandas as pd
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
```

PART-A

In [3]:
```
import tarfile
import pickle
import numpy as np

x_all, y_all = [], []

with tarfile.open("cifar-10-python.tar.gz", "r:gz") as tar:
    for i in range(1, 6):
        with tar.extractfile(
            f"cifar-10-batches-py/data_batch_{i}"
        ) as f:
            batch = pickle.load(f, encoding="bytes")
            x_all.append(batch[b"data"])
            y_all.extend(batch[b"labels"])

    with tar.extractfile("cifar-10-batches-py/test_batch") as f:
        batch = pickle.load(f, encoding="bytes")
        x_clean = batch[b"data"]
        y_clean = batch[b"labels"]

x_all = np.concatenate(x_all)
y_all = np.array(y_all).reshape(-1, 1)

x_all = x_all.reshape(-1, 3, 32, 32).transpose(0, 2, 3, 1)
x_clean = x_clean.reshape(-1, 3, 32, 32).transpose(0, 2, 3, 1)
y_clean = np.array(y_clean).reshape(-1, 1)

print(x_all.shape, y_all.shape)
print(x_clean.shape, y_clean.shape)
```

```
(50000, 32, 32, 3) (50000, 1)
(10000, 32, 32, 3) (10000, 1)
```

In [4]:
```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.keras.utils.set_random_seed(SEED)
```

In [5]:
```
y_all = y_all.flatten()
y_clean = y_clean.flatten()
print("Training Images:", x_all.shape)
print("Training Labels:", y_all.shape)
```

```
print("Clean Test Images:", x_clean.shape)
print("Clean Test Labels:", y_clean.shape)
```

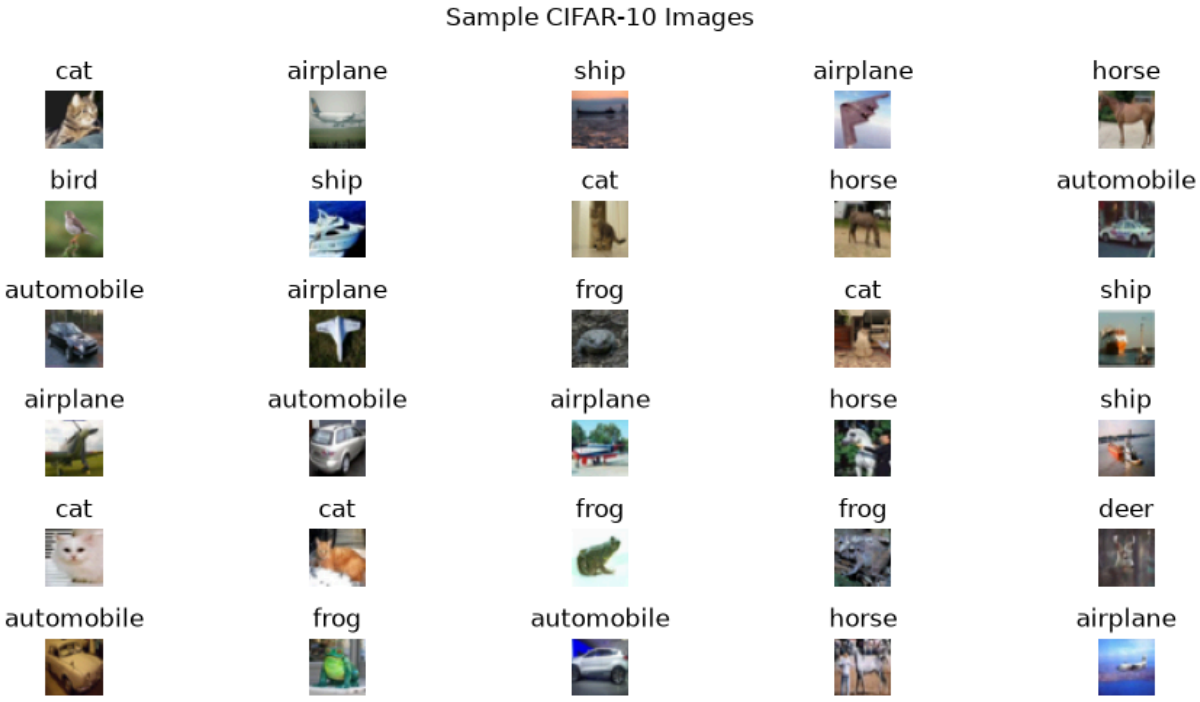
```
Training Images: (50000, 32, 32, 3)
Training Labels: (50000,)
Clean Test Images: (10000, 32, 32, 3)
Clean Test Labels: (10000,)
```

In [6]:
```
x_small, _, y_small, _ = train_test_split(
    x_all,
    y_all,
    train_size=500,
    random_state=SEED,
    stratify=y_all)
print("Small Dataset:", x_small.shape)
print("Labels:", y_small.shape)
```

```
Small Dataset: (500, 32, 32, 3)
Labels: (500,)
```

In [7]:
```
class_names = [
    "airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship"
]
plt.figure(figsize=(10, 5))
for i in range(30):
    plt.subplot(6, 5, i + 1)
    plt.imshow(x_small[i])
    plt.title(class_names[y_small[i]])
    plt.axis("off")

plt.suptitle("Sample CIFAR-10 Images")
plt.tight_layout()
plt.show()
```



In [8]:
```
x_train, x_val, y_train, y_val = train_test_split( x_small,y_small, test_size=100,
    random_state=SEED,
    stratify=y_small
)
print("Training Images:", x_train.shape)
print("Validation images:", x_val.shape)
```

```
Training Images: (400, 32, 32, 3)
Validation images: (100, 32, 32, 3)
```

In [9]:
```
def inception_module(x, filters=16):

    b1 = layers.Conv2D(
        filters, 1, padding="same", activation="relu"
    )(x)

    b3 = layers.Conv2D(
        filters, 1, padding="same", activation="relu"
    )(x)
    b3 = layers.Conv2D(
        filters, 3, padding="same", activation="relu"
    )(b3)

    b5 = layers.Conv2D(
        filters, 1, padding="same", activation="relu"
    )(x)
    b5 = layers.Conv2D(
        filters, 5, padding="same", activation="relu"
    )(b5)

    bp = layers.AveragePooling2D(
        pool_size=3, strides=1, padding="same"
    )(x)
    bp = layers.Conv2D(
        filters, 1, padding="same", activation="relu"
    )(bp)


    output = layers.Concatenate()([b1, b3, b5, bp])

    return output
```

In [10]:
```
inputs = keras.Input(shape=(32, 32, 3))

x = layers.Conv2D(
    24, 3, padding="same", activation="relu"
)(inputs)

x = layers.MaxPooling2D()(x)

x = layers.Conv2D(
    32, 3, padding="same", activation="relu"
)(x)

x = inception_module(x, filters=16)
```

```
x = layers.MaxPooling2D()(x)

x = layers.Conv2D(
    64, 3, padding="same", activation="relu"
)(x)

x = layers.GlobalAveragePooling2D()(x)



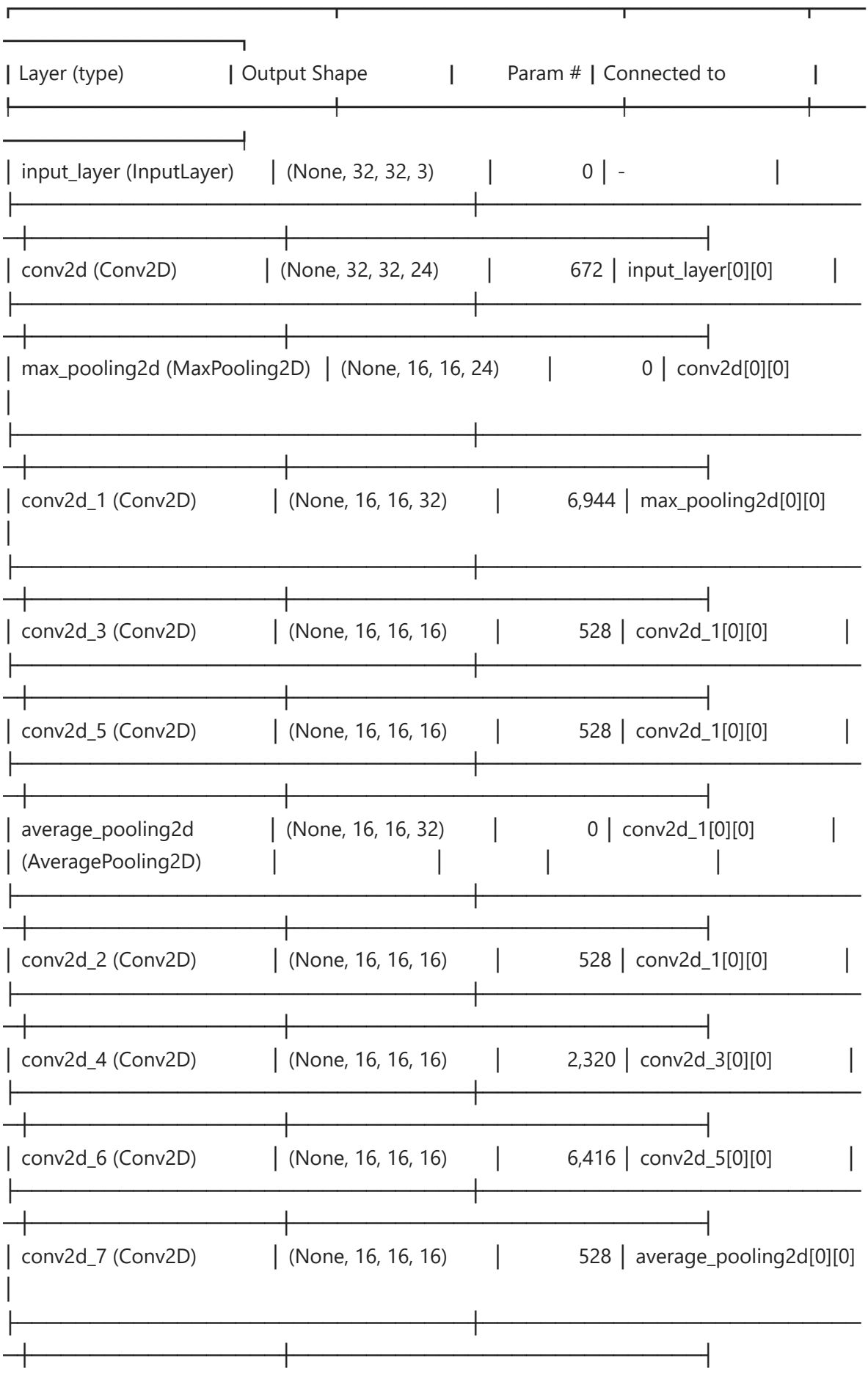
outputs = layers.Dense(
    10, activation="softmax"
)(x)

model = keras.Model(inputs, outputs)

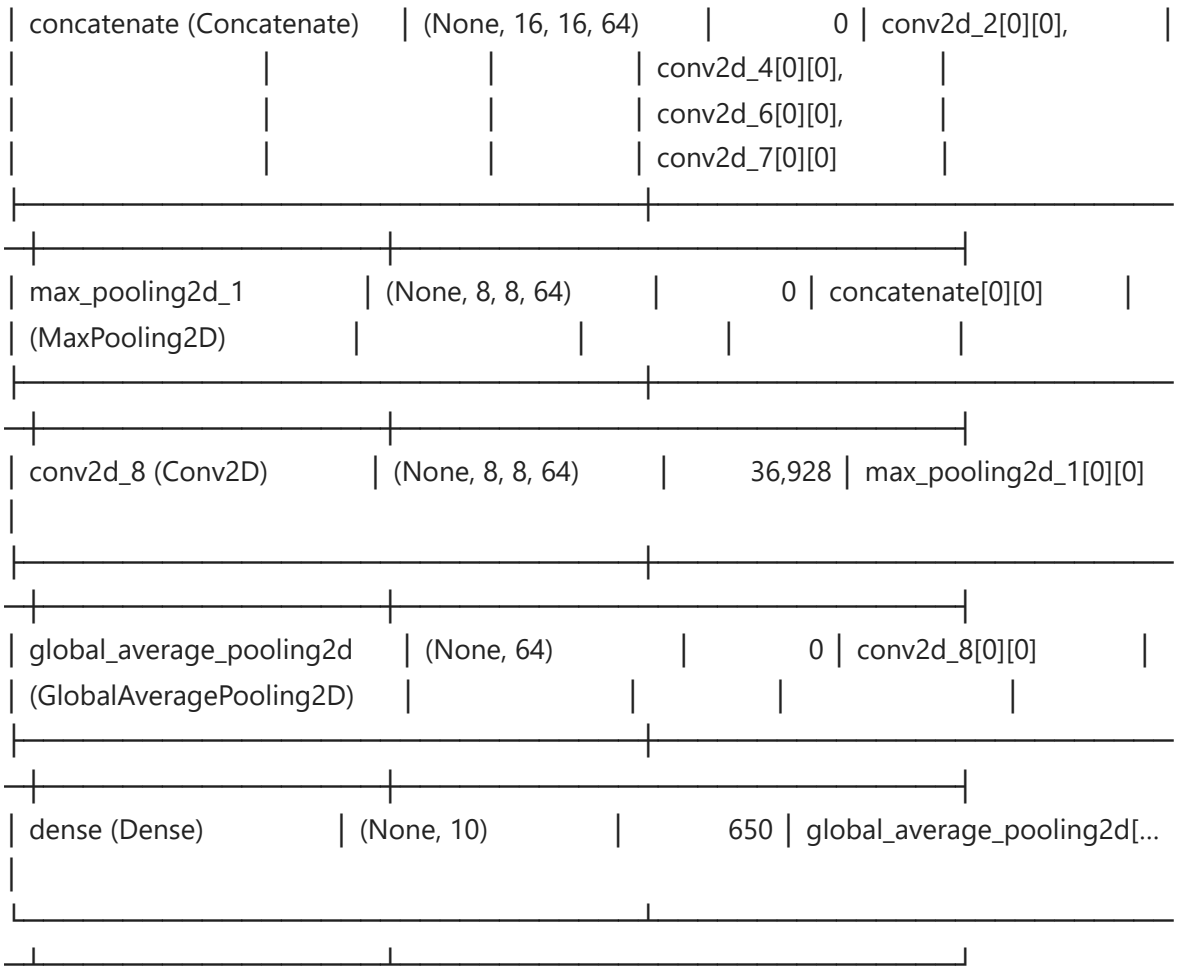
model.summary()

print("Total Parameters:", model.count_params())
```

Model: "functional"



Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 32, 32, 3)	0	-
conv2d (Conv2D)	(None, 32, 32, 24)	672	input_layer[0][0]
max_pooling2d (MaxPooling2D)	(None, 16, 16, 24)	0	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 16, 16, 32)	6,944	max_pooling2d[0][0]
conv2d_3 (Conv2D)	(None, 16, 16, 16)	528	conv2d_1[0][0]
conv2d_5 (Conv2D)	(None, 16, 16, 16)	528	conv2d_1[0][0]
average_pooling2d (AveragePooling2D)	(None, 16, 16, 32)	0	conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 16, 16, 16)	528	conv2d_1[0][0]
conv2d_4 (Conv2D)	(None, 16, 16, 16)	2,320	conv2d_3[0][0]
conv2d_6 (Conv2D)	(None, 16, 16, 16)	6,416	conv2d_5[0][0]
conv2d_7 (Conv2D)	(None, 16, 16, 16)	528	average_pooling2d[0][0]



Total params: 56,042 (218.91 KB)

Trainable params: 56,042 (218.91 KB)

Non-trainable params: 0 (0.00 B)

```
Total Parameters: 56042
```

In [11]:
```
model.compile(
    optimizer=keras.optimizers.Adam(
        learning_rate=0.001
    ),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

In [12]:
```
history = model.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 4s 153ms/step - accuracy: 0.0875 - loss: 8.5127 - val_acc
uracy: 0.1300 - val_loss: 3.1868
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.1100 - loss: 2.6463 - val_accu
racy: 0.1100 - val_loss: 2.3372
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.1400 - loss: 2.3253 - val_accu
racy: 0.1100 - val_loss: 2.2266
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.1975 - loss: 2.2181 - val_accu
racy: 0.2100 - val_loss: 2.1638
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.2625 - loss: 2.1317 - val_accu
racy: 0.2500 - val_loss: 2.0957
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.2825 - loss: 2.0301 - val_accu
racy: 0.2700 - val_loss: 2.0557
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.2825 - loss: 1.9321 - val_accu
racy: 0.2600 - val_loss: 2.0264
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3175 - loss: 1.8783 - val_accu
racy: 0.2500 - val_loss: 2.0171
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.3300 - loss: 1.8243 - val_accu
racy: 0.2400 - val_loss: 2.0561
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.3550 - loss: 1.7816 - val_accu
racy: 0.2500 - val_loss: 2.0423
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.3725 - loss: 1.7419 - val_accu
racy: 0.2600 - val_loss: 2.0827
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3500 - loss: 1.7246 - val_accu
racy: 0.2900 - val_loss: 2.0592
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3750 - loss: 1.6825 - val_accu
racy: 0.2800 - val_loss: 2.1415
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.4300 - loss: 1.5882 - val_accu
racy: 0.2900 - val_loss: 2.1014
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.4700 - loss: 1.4813 - val_accu
racy: 0.2900 - val_loss: 2.1265
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.5150 - loss: 1.4171 - val_accu
racy: 0.2400 - val_loss: 2.1756
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5300 - loss: 1.3488 - val_accu
racy: 0.2500 - val_loss: 2.1933
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.5500 - loss: 1.2862 - val_accu
racy: 0.2300 - val_loss: 2.2136
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.5675 - loss: 1.2489 - val_accu
```

```
racy: 0.2800 - val_loss: 2.2100
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.5800 - loss: 1.1856 - val_accu
racy: 0.3100 - val_loss: 2.1684
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.6025 - loss: 1.1059 - val_accu
racy: 0.3100 - val_loss: 2.2550
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.6350 - loss: 1.0866 - val_accu
racy: 0.3200 - val_loss: 2.2704
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6325 - loss: 1.0608 - val_accu
racy: 0.2800 - val_loss: 2.4347
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6075 - loss: 1.0703 - val_accu
racy: 0.2500 - val_loss: 2.3378
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6175 - loss: 1.0796 - val_accu
racy: 0.2200 - val_loss: 2.3263
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.6075 - loss: 1.1068 - val_accu
racy: 0.2200 - val_loss: 2.5777
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.6625 - loss: 0.9574 - val_accu
racy: 0.2600 - val_loss: 2.4117
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.6900 - loss: 0.9168 - val_accu
racy: 0.2500 - val_loss: 2.5692
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6975 - loss: 0.9197 - val_accu
racy: 0.2500 - val_loss: 2.4383
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6900 - loss: 0.8906 - val_accu
racy: 0.3000 - val_loss: 2.2755
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.6675 - loss: 0.8980 - val_accu
racy: 0.2900 - val_loss: 2.3437
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.6350 - loss: 0.9615 - val_accu
racy: 0.3100 - val_loss: 2.4686
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.6975 - loss: 0.8386 - val_accu
racy: 0.3100 - val_loss: 2.4897
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.7050 - loss: 0.8294 - val_accu
racy: 0.3000 - val_loss: 2.2060
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.7800 - loss: 0.6599 - val_accu
racy: 0.2800 - val_loss: 2.3123
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8075 - loss: 0.6199 - val_accu
racy: 0.2300 - val_loss: 2.5940
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8350 - loss: 0.5694 - val_accu
racy: 0.2700 - val_loss: 2.9358
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.7800 - loss: 0.6383 - val_accu
racy: 0.2400 - val_loss: 2.5952
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.7900 - loss: 0.6401 - val_accu
racy: 0.2700 - val_loss: 2.7493
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.7675 - loss: 0.6572 - val_accu
racy: 0.3400 - val_loss: 2.7063
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8000 - loss: 0.6469 - val_accu
racy: 0.3200 - val_loss: 2.6622
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8325 - loss: 0.5430 - val_accu
racy: 0.3400 - val_loss: 2.8058
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8775 - loss: 0.4259 - val_accu
racy: 0.3400 - val_loss: 2.8140
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8850 - loss: 0.4132 - val_accu
racy: 0.3100 - val_loss: 2.7532
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8700 - loss: 0.4172 - val_accu
racy: 0.4000 - val_loss: 2.7676
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8700 - loss: 0.4066 - val_accu
racy: 0.3600 - val_loss: 3.0466
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8750 - loss: 0.3927 - val_accu
racy: 0.2700 - val_loss: 3.3186
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.8900 - loss: 0.4084 - val_accu
racy: 0.3700 - val_loss: 3.0249
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8650 - loss: 0.4202 - val_accu
racy: 0.2700 - val_loss: 3.6190
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8350 - loss: 0.4751 - val_accu
racy: 0.3000 - val_loss: 2.7797
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.8300 - loss: 0.4762 - val_accu
racy: 0.3100 - val_loss: 3.1356
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.8425 - loss: 0.4686 - val_accu
racy: 0.3200 - val_loss: 2.9175
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8675 - loss: 0.3994 - val_accu
racy: 0.4000 - val_loss: 3.3268
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.8475 - loss: 0.4608 - val_accu
racy: 0.3200 - val_loss: 3.0555
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8625 - loss: 0.4065 - val_accu
racy: 0.3300 - val_loss: 3.4635
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.8800 - loss: 0.3991 - val_accu
racy: 0.2600 - val_loss: 3.0445
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.8950 - loss: 0.3548 - val_accu
racy: 0.3500 - val_loss: 3.3188
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8900 - loss: 0.3536 - val_accu
racy: 0.3200 - val_loss: 2.9774
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.9100 - loss: 0.3101 - val_accu
racy: 0.3400 - val_loss: 3.2559
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.9025 - loss: 0.3450 - val_accu
racy: 0.3700 - val_loss: 3.2437
```

In [13]:
```
train_loss, train_acc = model.evaluate(
    x_train, y_train, verbose=0
)

val_loss, val_acc = model.evaluate(
    x_val, y_val, verbose=0
)

print("Training Accuracy:", train_acc)
print("Validation Accuracy:", val_acc)
```

```
Training Accuracy: 0.8849999904632568
Validation Accuracy: 0.3700000047683716
```

In [14]:
```
clean_loss, clean_acc = model.evaluate(
    x_clean, y_clean, verbose=1
)

print("Clean Test Accuracy:", clean_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.3357 - loss: 3.3063
Clean Test Accuracy: 0.33570000529289246
```

In [15]:
```
print("Clean images:", x_clean.shape)
print("Clean labels:", y_clean.shape)
print("Image pixel range:", x_clean.min(), x_clean.max())
```

```
Clean images: (10000, 32, 32, 3)
Clean labels: (10000,)
Image pixel range: 0 255
```

In [16]:
```
x_shifted = x_clean.astype("float32").copy()

rotation = tf.keras.layers.RandomRotation(
    factor=25 / 360,
    fill_mode="nearest",
    seed=SEED
)

brightness = tf.keras.layers.RandomBrightness(
    factor=0.2,
    value_range=(0, 255),
    seed=SEED
)
```

```
contrast = tf.keras.layers.RandomContrast(
    factor=0.2,
    seed=SEED
)

x_shifted = rotation(x_shifted, training=True)
x_shifted = brightness(x_shifted, training=True)
x_shifted = contrast(x_shifted, training=True)

x_shifted = tf.clip_by_value(
    x_shifted, 0, 255
).numpy()

print("Shifted images:", x_shifted.shape)
print("Pixel range:", x_shifted.min(), x_shifted.max())
```

```
Shifted images: (10000, 32, 32, 3)
Pixel range: 0.0 255.0
```

In [17]:
```
rng = np.random.default_rng(SEED)

for i in range(len(x_shifted)):
    h, w, _ = x_shifted[i].shape

    box_h = h // 2
    box_w = w // 2

    top = rng.integers(0, h - box_h + 1)
    left = rng.integers(0, w - box_w + 1)

    x_shifted[
        i,
        top:top + box_h,
        left:left + box_w,
        :
    ] = 0

print("occlusion successfully")
```

```
occlusion successfully
```

In [18]:
```
y_shifted = y_clean.copy()

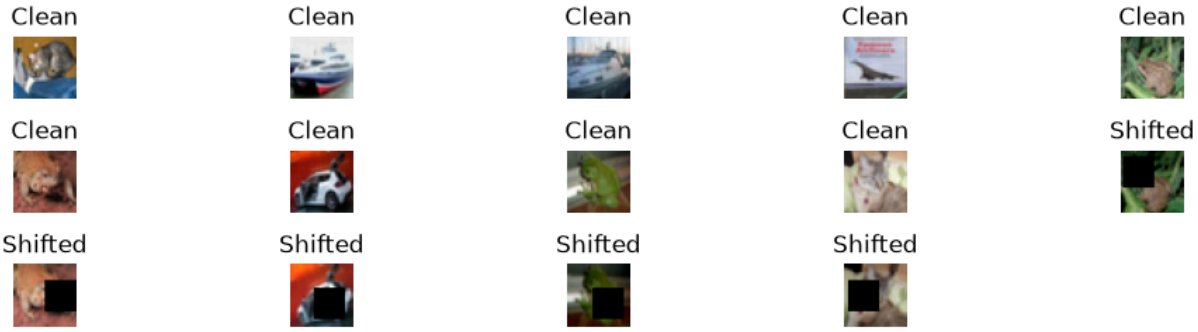
print("Shifted labels:", y_shifted.shape)
```

```
Shifted labels: (10000,)
```

In [19]:
```
class_names = [
    "airplane", "automobile", "bird", "cat", "deer","dog", "frog", "horse", "ship",
]
plt.figure(figsize=(10, 5))
for i in range(9):
    plt.subplot(6, 5, i + 1)
    plt.imshow(x_clean[i].astype("uint8"))
    plt.title("Clean")
    plt.axis("off")
```

```
    plt.subplot(6, 5, i + 6)
    plt.imshow(np.clip(x_shifted[i], 0, 255).astype("uint8"))
    plt.title("Shifted")
    plt.axis("off")

plt.tight_layout()
plt.show()
```



In [20]:
```
shifted_loss, shifted_acc = model.evaluate(
    x_shifted,
    y_shifted,
    verbose=1
)

print("Shifted Test Loss:", shifted_loss)
print("Shifted Test Accuracy:", shifted_acc * 100, "%")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.1843 - loss: 6.4325
Shifted Test Loss: 6.432494640350342
Shifted Test Accuracy: 18.43000054359436 %
```

In [21]:
```
x_shifted = x_shifted.astype("float32")

shifted_loss, shifted_acc = model.evaluate(
    x_shifted, y_shifted, verbose=1
)

print("Shifted Test Accuracy:", shifted_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.1843 - loss: 6.4325
Shifted Test Accuracy: 0.1843000054359436
```

In [22]:
```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title("Baseline Accuracy")
```

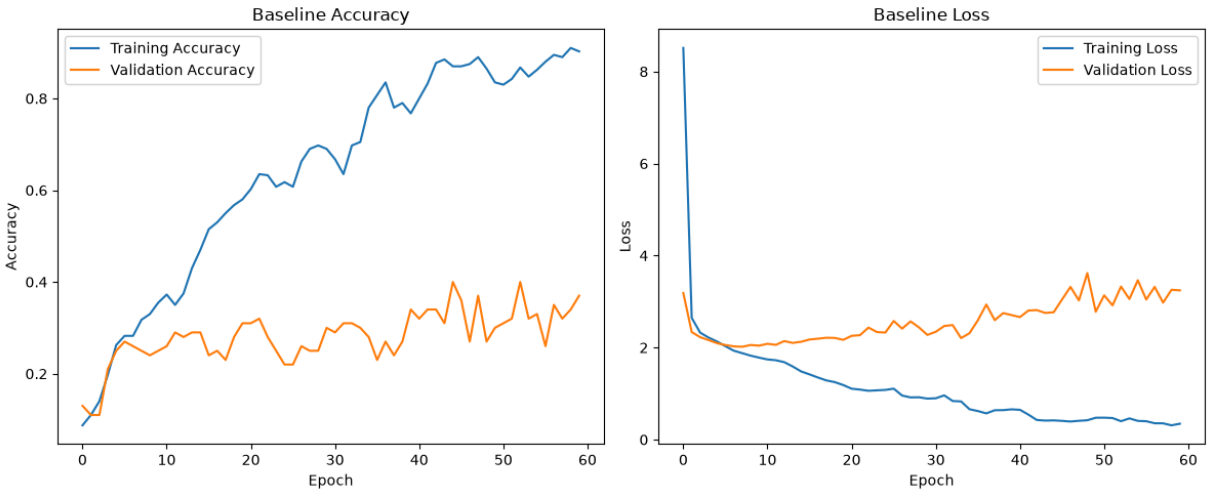
```
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()


plt.subplot(1, 2, 2)

plt.plot(
    history.history["loss"],
    label="Training Loss"
)
plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("Baseline Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()
```



SEED 2026

In [23]:
```
results = {
    "Dataset": [
        "Training",
        "Validation",
        "Clean Test",
        "Shifted Test"
    ],
    "Accuracy (%)": [
        train_acc * 100,
        val_acc * 100,
        clean_acc * 100,
        shifted_acc * 100
    ]
}
```

```
results_df = pd.DataFrame(results)

results_df["Accuracy (%)"] = results_df["Accuracy (%)"].round(2)

display(results_df)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Dataset</th> <th>Accuracy (%)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> </tbody> <tbody></tbody>

	Dataset	Accuracy (%)
0	Training	88.50
1	Validation	37.00
2	Clean Test	33.57
3	Shifted Test	18.43

The model learns from the training images. If training accuracy is much higher than validation accuracy it may be overfitting. Shifted images are harder to predict because they have changes like rotation, brightness, and noise. So, the model struggles with new image changes.

PART B

In [24]:
```
tf.keras.utils.set_random_seed(SEED)

data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),

    layers.RandomTranslation(
        height_factor=0.1,
        width_factor=0.1,
        fill_mode="nearest"
    ),

    layers.RandomRotation(
        factor=25 / 360,
        fill_mode="nearest"
    ),

    layers.RandomBrightness(
        factor=0.2,
        value_range=(0, 255)
    ),

    layers.RandomContrast(factor=0.2)
], name="data_augmentation")
```

In [25]:
```
inputs = keras.Input(shape=(32, 32, 3))

x = data_augmentation(inputs)
```

```
x = layers.Conv2D(
    24, 3, padding="same", activation="relu"
)(x)

x = layers.MaxPooling2D()(x)

x = layers.Conv2D(
    32, 3, padding="same", activation="relu"
)(x)

x = inception_module(x, filters=16)

x = layers.MaxPooling2D()(x)

x = layers.Conv2D(
    64, 3, padding="same", activation="relu"
)(x)

x = layers.GlobalAveragePooling2D()(x)

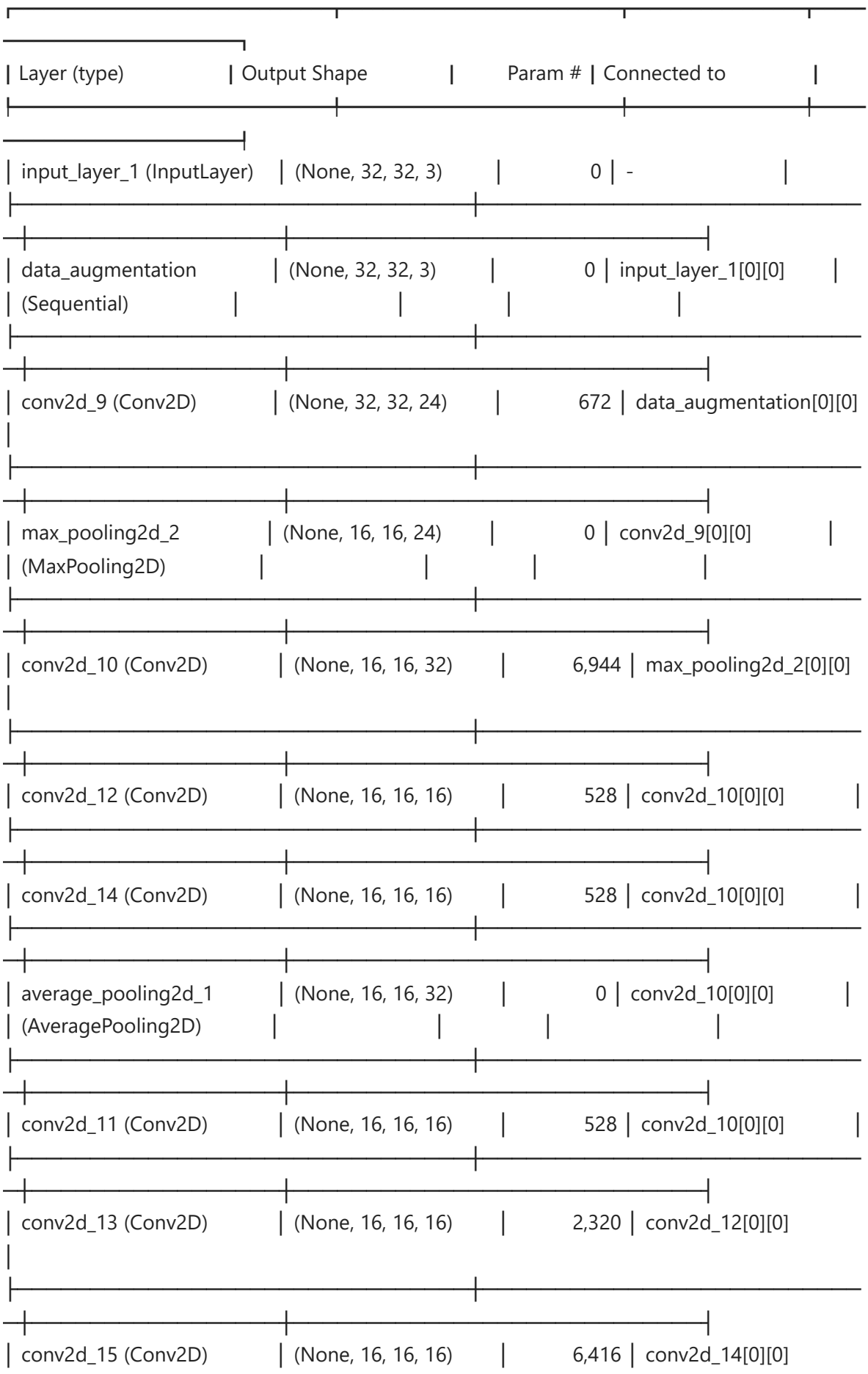
outputs = layers.Dense(
    10, activation="softmax"
)(x)

model_aug = keras.Model(inputs, outputs)

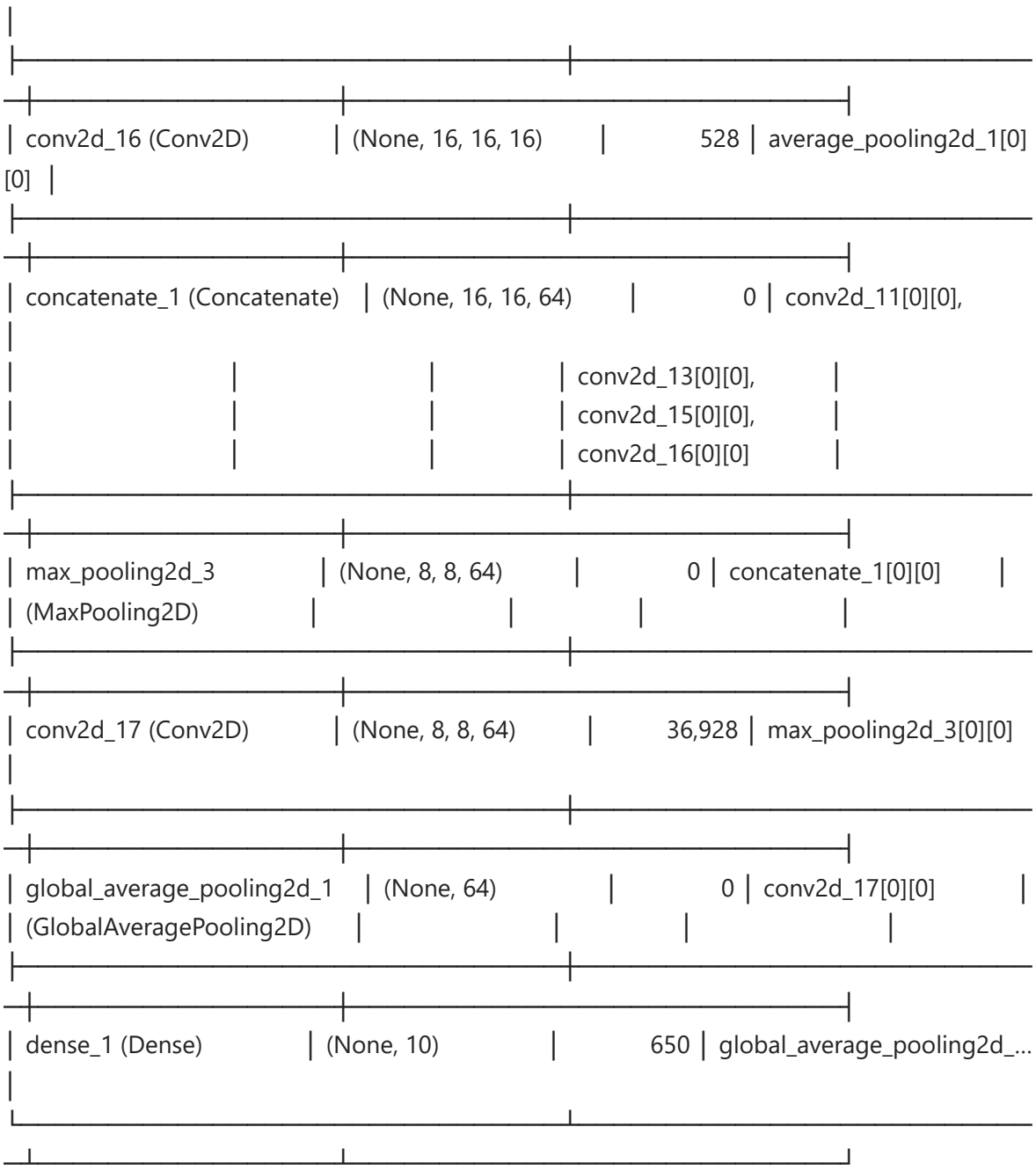
model_aug.summary()

print("Trainable parameters:",
      sum(np.prod(w.shape) for w in model_aug.trainable_weights))
```

Model: "functional_2"



Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 32, 32, 3)	0	-
data_augmentation (Sequential)	(None, 32, 32, 3)	0	input_layer_1[0][0]
conv2d_9 (Conv2D)	(None, 32, 32, 24)	672	data_augmentation[0][0]
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 24)	0	conv2d_9[0][0]
conv2d_10 (Conv2D)	(None, 16, 16, 32)	6,944	max_pooling2d_2[0][0]
conv2d_12 (Conv2D)	(None, 16, 16, 16)	528	conv2d_10[0][0]
conv2d_14 (Conv2D)	(None, 16, 16, 16)	528	conv2d_10[0][0]
average_pooling2d_1 (AveragePooling2D)	(None, 16, 16, 32)	0	conv2d_10[0][0]
conv2d_11 (Conv2D)	(None, 16, 16, 16)	528	conv2d_10[0][0]
conv2d_13 (Conv2D)	(None, 16, 16, 16)	2,320	conv2d_12[0][0]
conv2d_15 (Conv2D)	(None, 16, 16, 16)	6,416	conv2d_14[0][0]



Total params: 56,042 (218.91 KB)

Trainable params: 56,042 (218.91 KB)

Non-trainable params: 0 (0.00 B)

```
Trainable parameters: 56042
```

In [26]:
```
model_aug.compile(
    optimizer=keras.optimizers.Adam(
        learning_rate=0.001
    ),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

In [27]:
```
history_aug = model_aug.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 198ms/step - accuracy: 0.1200 - loss: 11.8056 - val_ac
curacy: 0.1100 - val_loss: 3.2047
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.1450 - loss: 2.6341 - val_accu
racy: 0.1600 - val_loss: 2.2535
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.1625 - loss: 2.2096 - val_accu
racy: 0.1500 - val_loss: 2.1873
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1475 - loss: 2.1964 - val_accu
racy: 0.1700 - val_loss: 2.1286
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.1725 - loss: 2.1598 - val_accu
racy: 0.1400 - val_loss: 2.1184
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.1925 - loss: 2.1177 - val_accu
racy: 0.1500 - val_loss: 2.0872
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2075 - loss: 2.0987 - val_accu
racy: 0.1500 - val_loss: 2.0747
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2050 - loss: 2.0751 - val_accu
racy: 0.1500 - val_loss: 2.0972
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.2500 - loss: 2.0386 - val_accu
racy: 0.1700 - val_loss: 2.1163
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.2600 - loss: 2.0178 - val_accu
racy: 0.2100 - val_loss: 2.0777
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3175 - loss: 1.9790 - val_accu
racy: 0.2600 - val_loss: 2.0547
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.2650 - loss: 1.9791 - val_accu
racy: 0.2300 - val_loss: 2.0636
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2500 - loss: 1.9438 - val_accu
racy: 0.1800 - val_loss: 2.0361
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2800 - loss: 1.9488 - val_accu
racy: 0.2200 - val_loss: 2.0472
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2900 - loss: 1.9588 - val_accu
racy: 0.2500 - val_loss: 2.0099
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2975 - loss: 1.9311 - val_accu
racy: 0.2400 - val_loss: 2.0600
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3175 - loss: 1.9066 - val_accu
racy: 0.2200 - val_loss: 2.0788
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2750 - loss: 1.9013 - val_accu
racy: 0.2300 - val_loss: 2.1099
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.3050 - loss: 1.9330 - val_accu
```

```
racy: 0.2400 - val_loss: 2.0547
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2925 - loss: 1.9437 - val_accu
racy: 0.2100 - val_loss: 2.1119
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.3300 - loss: 1.8877 - val_accu
racy: 0.2100 - val_loss: 2.0766
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3175 - loss: 1.9146 - val_accu
racy: 0.2400 - val_loss: 2.0508
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.2750 - loss: 1.9551 - val_accu
racy: 0.2800 - val_loss: 1.9772
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3050 - loss: 1.8511 - val_accu
racy: 0.2100 - val_loss: 1.9563
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3550 - loss: 1.8032 - val_accu
racy: 0.2400 - val_loss: 1.9418
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3550 - loss: 1.7656 - val_accu
racy: 0.2100 - val_loss: 2.0187
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3500 - loss: 1.8024 - val_accu
racy: 0.2300 - val_loss: 2.0468
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3150 - loss: 1.8347 - val_accu
racy: 0.2500 - val_loss: 1.9420
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3650 - loss: 1.7441 - val_accu
racy: 0.2900 - val_loss: 1.9592
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3600 - loss: 1.7260 - val_accu
racy: 0.2700 - val_loss: 1.9611
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4000 - loss: 1.6951 - val_accu
racy: 0.2900 - val_loss: 1.9656
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4000 - loss: 1.6952 - val_accu
racy: 0.3200 - val_loss: 1.9691
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4000 - loss: 1.6527 - val_accu
racy: 0.3600 - val_loss: 1.9353
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3850 - loss: 1.6556 - val_accu
racy: 0.3400 - val_loss: 1.9230
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4325 - loss: 1.5524 - val_accu
racy: 0.3000 - val_loss: 1.9274
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3925 - loss: 1.6346 - val_accu
racy: 0.3400 - val_loss: 1.9191
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4075 - loss: 1.6091 - val_accu
racy: 0.3200 - val_loss: 1.8866
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4300 - loss: 1.5487 - val_accu
racy: 0.3900 - val_loss: 1.9369
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4250 - loss: 1.5545 - val_accu
racy: 0.2800 - val_loss: 1.8786
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4325 - loss: 1.5637 - val_accu
racy: 0.3300 - val_loss: 1.9336
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.4100 - loss: 1.5997 - val_accu
racy: 0.3300 - val_loss: 1.8702
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4075 - loss: 1.5442 - val_accu
racy: 0.3800 - val_loss: 1.8961
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4100 - loss: 1.5474 - val_accu
racy: 0.4000 - val_loss: 1.8634
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4975 - loss: 1.4147 - val_accu
racy: 0.3600 - val_loss: 1.9092
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.5200 - loss: 1.4254 - val_accu
racy: 0.4200 - val_loss: 1.8663
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4925 - loss: 1.3846 - val_accu
racy: 0.4000 - val_loss: 2.0069
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4350 - loss: 1.5255 - val_accu
racy: 0.3100 - val_loss: 2.0635
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4675 - loss: 1.5161 - val_accu
racy: 0.3400 - val_loss: 1.9723
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4650 - loss: 1.4518 - val_accu
racy: 0.3600 - val_loss: 1.9106
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4825 - loss: 1.3818 - val_accu
racy: 0.3900 - val_loss: 1.9269
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.5325 - loss: 1.3541 - val_accu
racy: 0.3300 - val_loss: 1.9396
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.4750 - loss: 1.3939 - val_accu
racy: 0.3900 - val_loss: 1.9494
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.5075 - loss: 1.3392 - val_accu
racy: 0.4100 - val_loss: 2.0027
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.5000 - loss: 1.3762 - val_accu
racy: 0.3700 - val_loss: 1.9720
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - accuracy: 0.4925 - loss: 1.3799 - val_accu
racy: 0.4200 - val_loss: 2.0104
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.5000 - loss: 1.4098 - val_accu
racy: 0.3500 - val_loss: 1.8955
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.5225 - loss: 1.2951 - val_accu
racy: 0.3900 - val_loss: 1.9656
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.5200 - loss: 1.3157 - val_accu
racy: 0.3600 - val_loss: 2.0080
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.5375 - loss: 1.3051 - val_accu
racy: 0.3600 - val_loss: 2.0486
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.5450 - loss: 1.2503 - val_accu
racy: 0.3600 - val_loss: 2.0157
```

In [28]:
```
train_loss_aug, train_acc_aug = model_aug.evaluate(
    x_train, y_train, verbose=0
)

val_loss_aug, val_acc_aug = model_aug.evaluate(
    x_val, y_val, verbose=0
)

clean_loss_aug, clean_acc_aug = model_aug.evaluate(
    x_clean, y_clean, verbose=1
)

print("Training Accuracy:", train_acc_aug * 100, "%")
print("Validation Accuracy:", val_acc_aug * 100, "%")
print("Clean Test Accuracy:", clean_acc_aug * 100, "%")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.3759 - loss: 1.9743
Training Accuracy: 61.75000071525574 %
Validation Accuracy: 36.000001430511475 %
Clean Test Accuracy: 37.5900000333786 %
```

In [29]:
```
shifted_loss_aug, shifted_acc_aug = model_aug.evaluate(
    x_shifted, y_shifted, verbose=1
)

print("Synthetic Shifted Test Accuracy:",
      shifted_acc_aug * 100, "%")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.2028 - loss: 2.9087
Synthetic Shifted Test Accuracy: 20.28000056743622 %
```

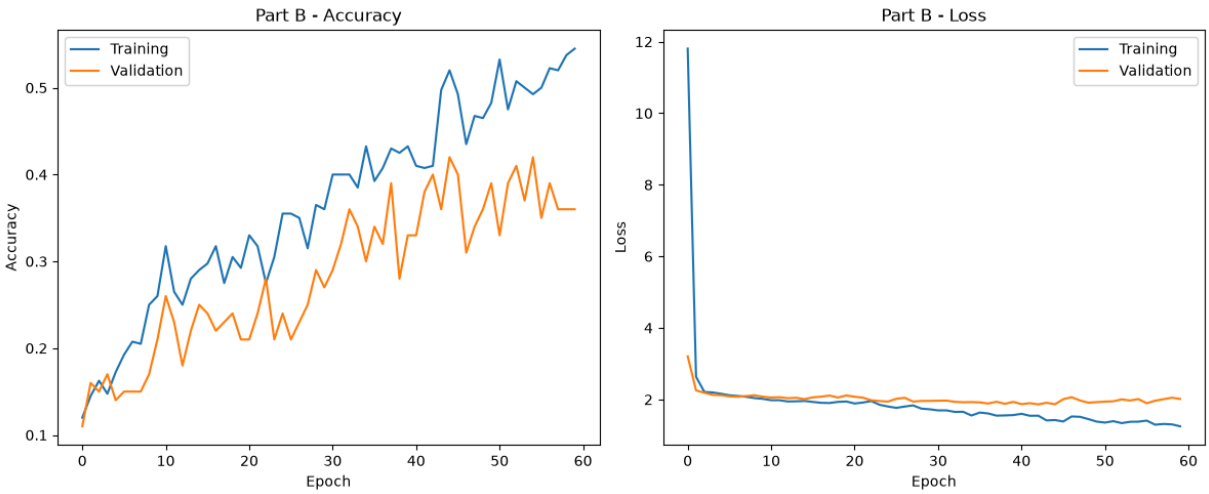
In [30]:
```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history_aug.history["accuracy"],
         label="Training")
plt.plot(history_aug.history["val_accuracy"],
         label="Validation")
plt.title("Part B - Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
```

```
plt.plot(history_aug.history["loss"],
         label="Training")
plt.plot(history_aug.history["val_loss"],
         label="Validation")
plt.title("Part B - Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()
```



<h2 id="Would-vertical-flip-help-or-hurt-on-this-dataset?-Why?">Would vertical flip help or hurt on this dataset? Why?</h2> Vertical flip may hurt accuracy because most of theobjects normally appear upright.Horizontal flip is more useful because it changes the left right direction without changing the objects class. <h2 id="If-you-augment-with-rotation-up-to-%C2%B190%C2%B0-but-the-test-only-has-%C2%B125%C2%B0,-what-is-the-trade-off?">If you augment with rotation up to ±90° but the test only has ±25°, what is the trade off?</h2> Large rotations may Improve performance on rotated imagebut they can also create unusual images and make training harder. Since test images rotate only up to ±25 smaller rotations are more suitable. <h2 id="Why-can-too-much-augmentation-reduce-accuracy-on-such-a-small-model?">Why can too much augmentation reduce accuracy on such a small model?</h2> Too much augmentation can make images too difficult or unrealistic. A small model trained on only 400 images may struggle to learn useful features. Therefore, we should use suitable augmentation and check its performance on validation data.

PART C

In [31]:
```
def conv_bn_relu(x, filters, kernel_size):
    x = layers.Conv2D(
        filters,
        kernel_size,
        padding="same",
        use_bias=False
    )(x)
```

```
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    return x


def inception_c(x, filters=16):

    b1 = conv_bn_relu(x, filters, 1)

    b3 = conv_bn_relu(x, filters, 1)
    b3 = conv_bn_relu(b3, filters, 3)

    b5 = conv_bn_relu(x, filters, 1)
    b5 = conv_bn_relu(b5, filters, 5)

    bp = layers.AveragePooling2D(
        pool_size=3, strides=1, padding="same"
    )(x)
    bp = conv_bn_relu(bp, filters, 1)

    return layers.Concatenate()([b1, b3, b5, bp])


inputs = keras.Input(shape=(32, 32, 3))

x = data_augmentation(inputs)

x = conv_bn_relu(x, 24, 3)
x = layers.MaxPooling2D()(x)

x = conv_bn_relu(x, 32, 3)
x = inception_c(x, filters=16)

x = layers.MaxPooling2D()(x)
x = conv_bn_relu(x, 64, 3)

x = layers.GlobalAveragePooling2D()(x)

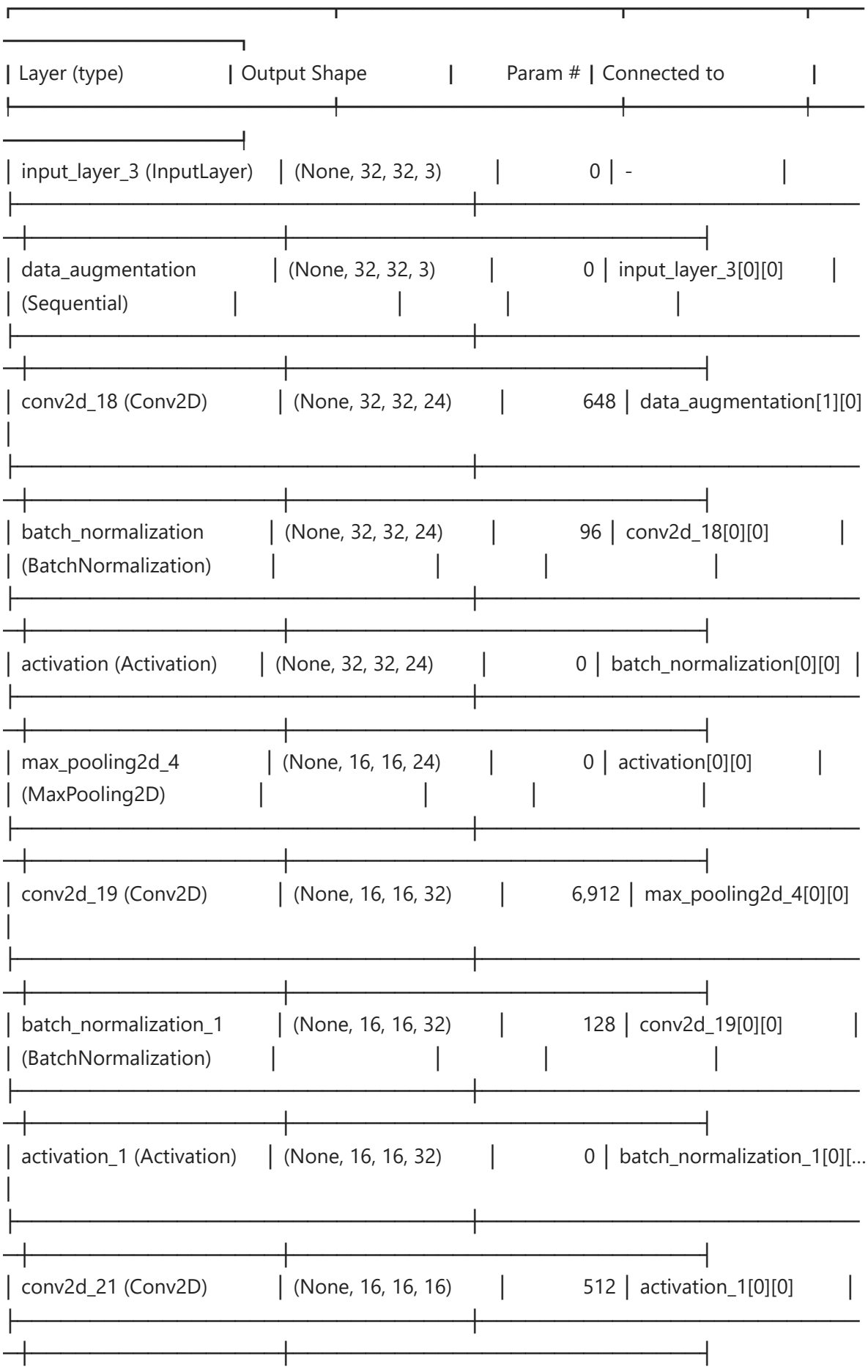
# Dropout
x = layers.Dropout(0.4)(x)

outputs = layers.Dense(10, activation="softmax")(x)

model_c = keras.Model(inputs, outputs)

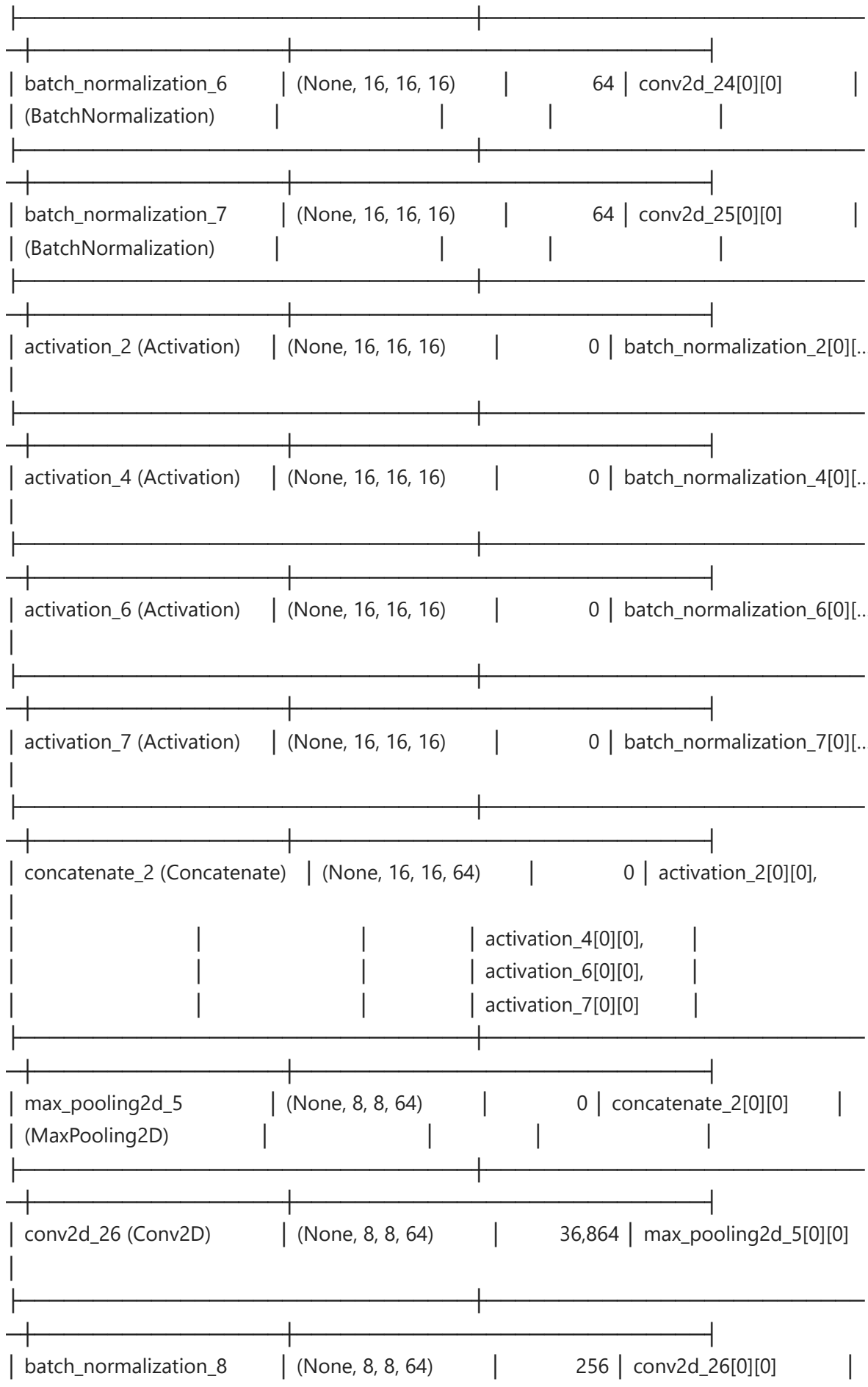
model_c.summary()
print("Trainable parameters:", model_c.count_params())
```

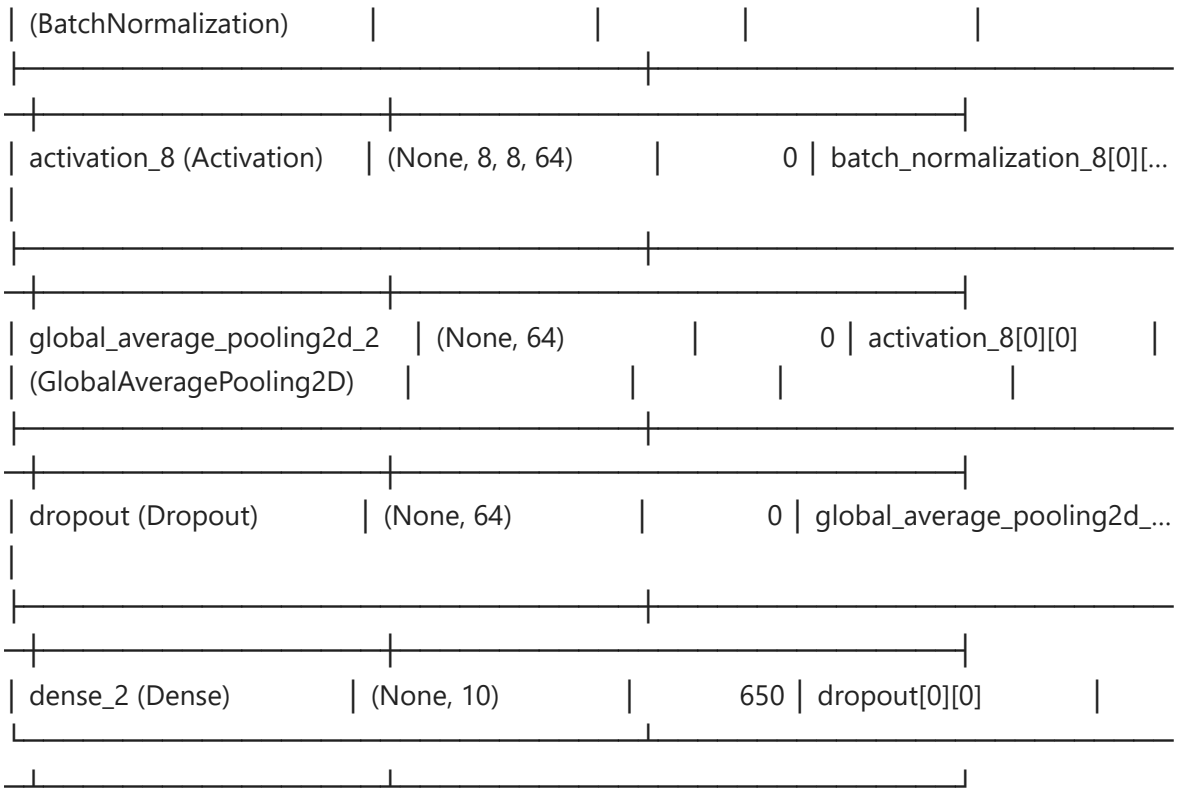
Model: "functional_3"



Layer (type)	Output Shape	Param #	Connected to
input_layer_3 (InputLayer)	(None, 32, 32, 3)	0	-
data_augmentation (Sequential)	(None, 32, 32, 3)	0	input_layer_3[0][0]
conv2d_18 (Conv2D)	(None, 32, 32, 24)	648	data_augmentation[1][0]
batch_normalization (BatchNormalization)	(None, 32, 32, 24)	96	conv2d_18[0][0]
activation (Activation)	(None, 32, 32, 24)	0	batch_normalization[0][0]
max_pooling2d_4 (MaxPooling2D)	(None, 16, 16, 24)	0	activation[0][0]
conv2d_19 (Conv2D)	(None, 16, 16, 32)	6,912	max_pooling2d_4[0][0]
batch_normalization_1 (BatchNormalization)	(None, 16, 16, 32)	128	conv2d_19[0][0]
activation_1 (Activation)	(None, 16, 16, 32)	0	batch_normalization_1[0][...
conv2d_21 (Conv2D)	(None, 16, 16, 16)	512	activation_1[0][0]

conv2d_23 (Conv2D)	(None, 16, 16, 16)	512	activation_1[0][0]
batch_normalization_3 (BatchNormalization)	(None, 16, 16, 16)	64	conv2d_21[0][0]
batch_normalization_5 (BatchNormalization)	(None, 16, 16, 16)	64	conv2d_23[0][0]
activation_3 (Activation)	(None, 16, 16, 16)	0	batch_normalization_3[0][...
activation_5 (Activation)	(None, 16, 16, 16)	0	batch_normalization_5[0][...
average_pooling2d_2 (AveragePooling2D)	(None, 16, 16, 32)	0	activation_1[0][0]
conv2d_20 (Conv2D)	(None, 16, 16, 16)	512	activation_1[0][0]
conv2d_22 (Conv2D)	(None, 16, 16, 16)	2,304	activation_3[0][0]
conv2d_24 (Conv2D)	(None, 16, 16, 16)	6,400	activation_5[0][0]
conv2d_25 (Conv2D)	(None, 16, 16, 16)	512	average_pooling2d_2[0][0]
batch_normalization_2 (BatchNormalization)	(None, 16, 16, 16)	64	conv2d_20[0][0]
batch_normalization_4 (BatchNormalization)	(None, 16, 16, 16)	64	conv2d_22[0][0]


batch_normalization_6 (BatchNormalization) (None, 16, 16, 16) 64 conv2d_24[0][0]
batch_normalization_7 (BatchNormalization) (None, 16, 16, 16) 64 conv2d_25[0][0]
activation_2 (Activation) (None, 16, 16, 16) 0 batch_normalization_2[0][...
activation_4 (Activation) (None, 16, 16, 16) 0 batch_normalization_4[0][...
activation_6 (Activation) (None, 16, 16, 16) 0 batch_normalization_6[0][...
activation_7 (Activation) (None, 16, 16, 16) 0 batch_normalization_7[0][...
concatenate_2 (Concatenate) (None, 16, 16, 64) 0 activation_2[0][0], activation_4[0][0], activation_6[0][0], activation_7[0][0]
max_pooling2d_5 (MaxPooling2D) (None, 8, 8, 64) 0 concatenate_2[0][0]
conv2d_26 (Conv2D) (None, 8, 8, 64) 36,864 max_pooling2d_5[0][0]
batch_normalization_8 (None, 8, 8, 64) 256 conv2d_26[0][0]



Total params: 56,690 (221.45 KB)

Trainable params: 56,258 (219.76 KB)

Non-trainable params: 432 (1.69 KB)

```
Trainable parameters: 56690
```

In [32]:
```
y_train_onehot = keras.utils.to_categorical(y_train, 10)
y_val_onehot = keras.utils.to_categorical(y_val, 10)
```

In [33]:
```
model_c.compile(
    optimizer=keras.optimizers.AdamW(
        learning_rate=0.001,
        weight_decay=1e-4
    ),
    loss=keras.losses.CategoricalCrossentropy(
        label_smoothing=0.05
    ),
    metrics=["accuracy"]
)
```

In [34]:
```
callbacks_c = [

    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=8,
        restore_best_weights=True,
        verbose=1
    )
]
```

In [35]:
```
history_c = model_c.fit(
    x_train,
    y_train_onehot,
    validation_data=(x_val, y_val_onehot),
    epochs=60,
    batch_size=32,
    callbacks=callbacks_c,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 9s 389ms/step - accuracy: 0.1225 - loss: 2.4669 - val_acc
uracy: 0.1000 - val_loss: 3.6122
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.2100 - loss: 2.2216 - val_accu
racy: 0.1200 - val_loss: 3.3180
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 55ms/step - accuracy: 0.2200 - loss: 2.1640 - val_accu
racy: 0.1300 - val_loss: 2.9193
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.2250 - loss: 2.0979 - val_accu
racy: 0.1100 - val_loss: 2.6425
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - accuracy: 0.2700 - loss: 2.0373 - val_accu
racy: 0.1000 - val_loss: 2.9019
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.2575 - loss: 1.9691 - val_accu
racy: 0.1600 - val_loss: 2.2490
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.2925 - loss: 1.9464 - val_accu
racy: 0.2300 - val_loss: 2.1687
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.3350 - loss: 1.9203 - val_accu
racy: 0.1600 - val_loss: 2.3637
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.3325 - loss: 1.9095 - val_accu
racy: 0.1700 - val_loss: 2.4641
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.3125 - loss: 1.8795 - val_accu
racy: 0.2100 - val_loss: 2.4158
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.3700 - loss: 1.8111 - val_accu
racy: 0.2700 - val_loss: 2.2363
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.3400 - loss: 1.8227 - val_accu
racy: 0.2100 - val_loss: 2.4035
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 55ms/step - accuracy: 0.3900 - loss: 1.7676 - val_accu
racy: 0.2300 - val_loss: 2.2745
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 55ms/step - accuracy: 0.4025 - loss: 1.7536 - val_accu
racy: 0.2400 - val_loss: 2.2501
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.3775 - loss: 1.7752 - val_accu
racy: 0.1800 - val_loss: 2.5002
Epoch 15: early stopping
Restoring model weights from the end of the best epoch: 7.
```

In [36]:

```
train_loss_c, train_acc_c = model_c.evaluate(
    x_train, y_train_onehot, verbose=0
)

val_loss_c, val_acc_c = model_c.evaluate(
    x_val, y_val_onehot, verbose=0
)
```

```
clean_loss_c, clean_acc_c = model_c.evaluate(
    x_clean, keras.utils.to_categorical(y_clean, 10),
    verbose=1
)

print("Training Accuracy:", train_acc_c * 100, "%")
print("Validation Accuracy:", val_acc_c * 100, "%")
print("Clean Test Accuracy:", clean_acc_c * 100, "%")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.2142 - loss: 2.1510
Training Accuracy: 25.24999976158142 %
Validation Accuracy: 23.000000417232513 %
Clean Test Accuracy: 21.42000049352646 %
```

In [37]:
```
y_shifted_onehot = keras.utils.to_categorical(y_shifted, 10)

shifted_loss_c, shifted_acc_c = model_c.evaluate(
    x_shifted, y_shifted_onehot, verbose=1
)

print("Synthetic Shifted Accuracy:", shifted_acc_c * 100, "%")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step - accuracy: 0.1232 - loss: 2.4567
Synthetic Shifted Accuracy: 12.319999933242798 %
```

<h3 id="Regularization-and-Training-Design">Regularization and Training Design</h3>
Batch Normalization is used after convolutional layers to make training more stable. Dropout is added after Global Average Pooling to help prevent overfitting. AdamW uses weight decay to prevent the model's weights from becoming too large. Label smoothing helps the model make less overconfident predictions. EarlyStopping stops training when the validation loss does not improve and restores the best model weights.

PART D

In [38]:
```
def build_model(augmentation=None):
    inputs = keras.Input(shape=(32, 32, 3))

    x = inputs

    if augmentation is not None:
        x = augmentation(x)

    x = layers.Conv2D(
        24, 3, padding="same", activation="relu"
    )(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(
        32, 3, padding="same", activation="relu"
    )(x)

    x = inception_module(x, filters=16)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(
```

```
        64, 3, padding="same", activation="relu"
    )(x)

    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.3)(x)

    outputs = layers.Dense(
        10, activation="softmax"
    )(x)

    model = keras.Model(inputs, outputs)

    model.compile(
        optimizer=keras.optimizers.Adam(0.001),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

    return model
```

In [39]:

```
geometric_aug = keras.Sequential([
    layers.RandomTranslation(0.1, 0.1),
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(25 / 360)
])

tf.keras.utils.set_random_seed(42)

model_geo = build_model(geometric_aug)

history_geo = model_geo.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)

val_loss, val_acc = model_geo.evaluate(
    x_val, y_val, verbose=0
)

print("Geometric validation accuracy:", val_acc)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 187ms/step - accuracy: 0.1150 - loss: 5.6301 - val_acc
uracy: 0.1500 - val_loss: 2.2909
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.1300 - loss: 2.2950 - val_accu
racy: 0.1600 - val_loss: 2.2667
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.1600 - loss: 2.2474 - val_accu
racy: 0.1500 - val_loss: 2.2134
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.1450 - loss: 2.2259 - val_accu
racy: 0.1700 - val_loss: 2.2158
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.1500 - loss: 2.2187 - val_accu
racy: 0.2100 - val_loss: 2.1721
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.1850 - loss: 2.1942 - val_accu
racy: 0.1600 - val_loss: 2.1943
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.2025 - loss: 2.1576 - val_accu
racy: 0.2200 - val_loss: 2.1446
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.1775 - loss: 2.1288 - val_accu
racy: 0.2400 - val_loss: 2.1445
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1900 - loss: 2.1122 - val_accu
racy: 0.2300 - val_loss: 2.1174
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.2175 - loss: 2.0764 - val_accu
racy: 0.2200 - val_loss: 2.1595
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2675 - loss: 2.0668 - val_accu
racy: 0.1900 - val_loss: 2.1421
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.2325 - loss: 2.0505 - val_accu
racy: 0.2300 - val_loss: 2.0678
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.2575 - loss: 1.9931 - val_accu
racy: 0.2100 - val_loss: 2.0734
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.2300 - loss: 1.9980 - val_accu
racy: 0.2400 - val_loss: 2.0813
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2600 - loss: 1.9966 - val_accu
racy: 0.2500 - val_loss: 1.9977
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.2650 - loss: 1.9942 - val_accu
racy: 0.2900 - val_loss: 2.0312
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.2625 - loss: 1.9365 - val_accu
racy: 0.2500 - val_loss: 1.9887
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.2575 - loss: 1.9049 - val_accu
racy: 0.2500 - val_loss: 1.9648
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.2925 - loss: 1.8954 - val_accu
```

```
racy: 0.2800 - val_loss: 1.9481
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3200 - loss: 1.8840 - val_accu
racy: 0.2900 - val_loss: 1.9708
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.3175 - loss: 1.8126 - val_accu
racy: 0.3100 - val_loss: 1.9455
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2975 - loss: 1.9174 - val_accu
racy: 0.2800 - val_loss: 2.0412
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3075 - loss: 1.8954 - val_accu
racy: 0.2500 - val_loss: 1.9455
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2900 - loss: 1.8656 - val_accu
racy: 0.2800 - val_loss: 1.9619
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2675 - loss: 1.8896 - val_accu
racy: 0.2700 - val_loss: 1.9243
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2975 - loss: 1.8600 - val_accu
racy: 0.2700 - val_loss: 2.0680
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3175 - loss: 1.8073 - val_accu
racy: 0.3300 - val_loss: 2.0375
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3100 - loss: 1.7996 - val_accu
racy: 0.3300 - val_loss: 1.9587
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.3250 - loss: 1.8009 - val_accu
racy: 0.2400 - val_loss: 2.0148
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3175 - loss: 1.8189 - val_accu
racy: 0.3300 - val_loss: 2.0073
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3100 - loss: 1.8221 - val_accu
racy: 0.3000 - val_loss: 2.0408
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3425 - loss: 1.8323 - val_accu
racy: 0.3200 - val_loss: 2.0485
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3750 - loss: 1.7158 - val_accu
racy: 0.3700 - val_loss: 1.8919
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3850 - loss: 1.6869 - val_accu
racy: 0.3000 - val_loss: 1.9480
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3700 - loss: 1.7507 - val_accu
racy: 0.2800 - val_loss: 1.9245
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3550 - loss: 1.7398 - val_accu
racy: 0.3300 - val_loss: 1.8975
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3650 - loss: 1.7040 - val_accu
racy: 0.3100 - val_loss: 2.0125
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.3525 - loss: 1.6446 - val_accu
racy: 0.3300 - val_loss: 2.0734
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3625 - loss: 1.6785 - val_accu
racy: 0.3300 - val_loss: 2.1703
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3850 - loss: 1.6490 - val_accu
racy: 0.3300 - val_loss: 2.0005
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3675 - loss: 1.6688 - val_accu
racy: 0.3100 - val_loss: 2.0584
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3925 - loss: 1.6899 - val_accu
racy: 0.3100 - val_loss: 2.1077
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.3600 - loss: 1.7079 - val_accu
racy: 0.3100 - val_loss: 2.1198
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3950 - loss: 1.5955 - val_accu
racy: 0.3300 - val_loss: 2.1280
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4175 - loss: 1.6205 - val_accu
racy: 0.3700 - val_loss: 1.9853
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4175 - loss: 1.5796 - val_accu
racy: 0.3200 - val_loss: 1.9473
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4425 - loss: 1.5635 - val_accu
racy: 0.3300 - val_loss: 2.0447
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4050 - loss: 1.5852 - val_accu
racy: 0.2900 - val_loss: 2.1828
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4300 - loss: 1.5475 - val_accu
racy: 0.3400 - val_loss: 2.0811
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4175 - loss: 1.5078 - val_accu
racy: 0.3200 - val_loss: 2.0516
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4375 - loss: 1.4954 - val_accu
racy: 0.3000 - val_loss: 2.1938
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4825 - loss: 1.5103 - val_accu
racy: 0.3600 - val_loss: 2.2115
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4200 - loss: 1.5146 - val_accu
racy: 0.3000 - val_loss: 2.2710
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4350 - loss: 1.5461 - val_accu
racy: 0.3700 - val_loss: 2.0571
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4650 - loss: 1.4892 - val_accu
racy: 0.3400 - val_loss: 2.2891
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4600 - loss: 1.4540 - val_accu
racy: 0.3700 - val_loss: 2.0552
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4575 - loss: 1.5405 - val_accu
racy: 0.3800 - val_loss: 2.0117
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4125 - loss: 1.5443 - val_accu
racy: 0.3600 - val_loss: 1.9754
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4325 - loss: 1.5147 - val_accu
racy: 0.3600 - val_loss: 2.0447
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.4275 - loss: 1.5128 - val_accu
racy: 0.3900 - val_loss: 2.0358
Geometric validation accuracy: 0.38999998569488525
```

In [40]:
```
results = [
    "No augmentation",
    "Geometric only",
    "Photometric only",
    "Geometric + photometric",
    "+ Cutout / Random Erasing",
    "+ Mixup / CutMix + label smoothing",
    "Final model"
]

df_results = pd.DataFrame({
    "Configuration": results,
    "Val Acc": [None] * 7,
    "Clean Test": [None] * 7,
    "Shifted Test (Mean ± Std)": [None] * 7
})

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

	Configuration	Val Acc	Clean Test	Shifted Test (Mean ± Std)
0	No augmentation	None	None	None
1	Geometric only	None	None	None
2	Photometric only	None	None	None
3	Geometric + photometric	None	None	None
4	+ Cutout / Random Erasing	None	None	None
5	+ Mixup / CutMix + label smoothing	None	None	None
6	Final model	None	None	None

In [41]:
```
_, val_acc = model_geo.evaluate(
    x_val, y_val, verbose=0
)
```

```
_, clean_acc = model_geo.evaluate(
    x_clean, y_clean, verbose=0
)

print("Validation Accuracy:", round(val_acc * 100, 2), "%")
print("Clean Test Accuracy:", round(clean_acc * 100, 2), "%")
```

```
Validation Accuracy: 39.0 %
Clean Test Accuracy: 37.34 %
```

In [42]:
```
_, shifted_acc = model_geo.evaluate(
    x_shifted, y_shifted, verbose=0
)

print("Shifted Test Accuracy:",
      round(shifted_acc * 100, 2), "%")
```

```
Shifted Test Accuracy: 20.03 %
```

In [43]:
```
_, shifted_acc_geo = model_geo.evaluate(
    x_shifted,
    y_shifted,
    verbose=0
)

print("Shifted accuracy:", shifted_acc_geo)
print("Shifted accuracy (%):", shifted_acc_geo * 100)
```

```
Shifted accuracy: 0.20029999315738678
Shifted accuracy (%): 20.029999315738678
```

In [44]:
```
df_results.loc[1, "Val Acc"] = round(
    val_acc * 100, 2
)

df_results.loc[1, "Clean Test"] = round(
    clean_acc * 100, 2
)
df_results.loc[1, "Shifted Test (Mean ± Std)"] = (
    f"{shifted_acc_geo * 100:.2f}% (seed 42)"
)
display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	None	None	None
Geometric only	39.0	37.34	20.03% (seed 42)
Photometric only	None	None	None
Geometric + photometric	None	None	None
+ Cutout / Random Erasing	None	None	None
+ Mixup / CutMix + label smoothing	None	None	None
Final model	None	None	None

In [45]:
```
_, shifted_acc_geo = model_geo.evaluate(
    x_shifted,
    y_shifted,
    verbose=0
)

print("Shifted accuracy:", shifted_acc_geo)
print("Shifted accuracy (%):", shifted_acc_geo * 100)
```

```
Shifted accuracy: 0.20029999315738678
Shifted accuracy (%): 20.029999315738678
```

In [46]:
```
tf.keras.utils.set_random_seed(SEED)

photometric_aug = keras.Sequential([
    layers.RandomBrightness(
        factor=0.2,
        value_range=(0, 255)
    ),
    layers.RandomContrast(factor=0.2)
])
```

In [47]:
```
model_photo = build_model(photometric_aug)

history_photo = model_photo.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 4s 173ms/step - accuracy: 0.1025 - loss: 7.5603 - val_acc
uracy: 0.1500 - val_loss: 2.3371
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.1250 - loss: 2.3303 - val_accu
racy: 0.1900 - val_loss: 2.2895
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.1550 - loss: 2.2780 - val_accu
racy: 0.1300 - val_loss: 2.2673
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.1600 - loss: 2.2434 - val_accu
racy: 0.1500 - val_loss: 2.2497
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.1750 - loss: 2.2288 - val_accu
racy: 0.1900 - val_loss: 2.2181
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.1850 - loss: 2.2062 - val_accu
racy: 0.1000 - val_loss: 2.2005
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2075 - loss: 2.1790 - val_accu
racy: 0.1900 - val_loss: 2.1801
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.1900 - loss: 2.1398 - val_accu
racy: 0.1600 - val_loss: 2.1347
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.1950 - loss: 2.0843 - val_accu
racy: 0.2200 - val_loss: 2.1218
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.2175 - loss: 2.0937 - val_accu
racy: 0.2500 - val_loss: 2.1096
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2350 - loss: 2.0850 - val_accu
racy: 0.2200 - val_loss: 2.0814
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.2575 - loss: 2.0164 - val_accu
racy: 0.2400 - val_loss: 2.1117
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2725 - loss: 1.9783 - val_accu
racy: 0.2000 - val_loss: 2.0979
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.2600 - loss: 2.0457 - val_accu
racy: 0.2200 - val_loss: 2.0799
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.2725 - loss: 2.0161 - val_accu
racy: 0.2800 - val_loss: 2.1085
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.2975 - loss: 1.9580 - val_accu
racy: 0.2500 - val_loss: 2.1041
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.3225 - loss: 1.9336 - val_accu
racy: 0.2400 - val_loss: 2.0991
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.2575 - loss: 1.9550 - val_accu
racy: 0.2200 - val_loss: 2.1020
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.2900 - loss: 1.8821 - val_accu
```

```
racy: 0.2200 - val_loss: 2.0852
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3300 - loss: 1.8170 - val_accu
racy: 0.2100 - val_loss: 2.0939
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3675 - loss: 1.7898 - val_accu
racy: 0.2300 - val_loss: 2.0449
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.3400 - loss: 1.7667 - val_accu
racy: 0.2300 - val_loss: 2.0331
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.3625 - loss: 1.7204 - val_accu
racy: 0.1900 - val_loss: 2.0884
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.3600 - loss: 1.7430 - val_accu
racy: 0.1800 - val_loss: 1.9972
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3950 - loss: 1.6672 - val_accu
racy: 0.2400 - val_loss: 2.0265
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.4075 - loss: 1.6745 - val_accu
racy: 0.2600 - val_loss: 1.9947
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.3975 - loss: 1.6192 - val_accu
racy: 0.2200 - val_loss: 2.0340
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4075 - loss: 1.6347 - val_accu
racy: 0.2700 - val_loss: 2.1163
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4575 - loss: 1.5860 - val_accu
racy: 0.3100 - val_loss: 1.9033
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4550 - loss: 1.5326 - val_accu
racy: 0.2900 - val_loss: 2.0274
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.4475 - loss: 1.5406 - val_accu
racy: 0.2700 - val_loss: 1.9021
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4725 - loss: 1.4335 - val_accu
racy: 0.2800 - val_loss: 1.9253
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4650 - loss: 1.4410 - val_accu
racy: 0.2700 - val_loss: 1.9974
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.4750 - loss: 1.4135 - val_accu
racy: 0.2400 - val_loss: 2.0420
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5125 - loss: 1.3568 - val_accu
racy: 0.3100 - val_loss: 1.9110
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4925 - loss: 1.3991 - val_accu
racy: 0.2900 - val_loss: 2.0477
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5200 - loss: 1.3122 - val_accu
racy: 0.3200 - val_loss: 2.0396
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5325 - loss: 1.2954 - val_accu
racy: 0.3700 - val_loss: 2.0583
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5325 - loss: 1.2774 - val_accu
racy: 0.3300 - val_loss: 2.1145
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5800 - loss: 1.2282 - val_accu
racy: 0.2900 - val_loss: 1.9384
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5225 - loss: 1.2580 - val_accu
racy: 0.3300 - val_loss: 1.9802
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.5875 - loss: 1.1579 - val_accu
racy: 0.3300 - val_loss: 1.9815
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5675 - loss: 1.2414 - val_accu
racy: 0.3100 - val_loss: 1.8770
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5825 - loss: 1.1653 - val_accu
racy: 0.3700 - val_loss: 1.9614
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5875 - loss: 1.1092 - val_accu
racy: 0.3200 - val_loss: 2.0519
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.6150 - loss: 1.0824 - val_accu
racy: 0.3100 - val_loss: 1.8721
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6175 - loss: 1.0392 - val_accu
racy: 0.3400 - val_loss: 2.0079
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6550 - loss: 0.9864 - val_accu
racy: 0.3600 - val_loss: 2.0917
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6600 - loss: 0.9760 - val_accu
racy: 0.3800 - val_loss: 1.9620
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6400 - loss: 1.0208 - val_accu
racy: 0.3300 - val_loss: 2.1063
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.6550 - loss: 1.0034 - val_accu
racy: 0.3500 - val_loss: 2.3933
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6575 - loss: 0.9720 - val_accu
racy: 0.3300 - val_loss: 1.9666
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.6825 - loss: 0.9015 - val_accu
racy: 0.3300 - val_loss: 2.1120
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.7125 - loss: 0.8223 - val_accu
racy: 0.3600 - val_loss: 1.9997
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6825 - loss: 0.8965 - val_accu
racy: 0.3800 - val_loss: 2.1749
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.6700 - loss: 0.9109 - val_accu
racy: 0.3800 - val_loss: 2.0437
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6925 - loss: 0.8836 - val_accu
racy: 0.4000 - val_loss: 2.0591
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.7300 - loss: 0.8006 - val_accu
racy: 0.3800 - val_loss: 2.3946
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.7000 - loss: 0.8165 - val_accu
racy: 0.3300 - val_loss: 2.3664
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6775 - loss: 0.8692 - val_accu
racy: 0.3000 - val_loss: 2.2005
```

In [48]:
```
_, val_acc = model_photo.evaluate(
    x_val, y_val, verbose=0
)

_, clean_acc = model_photo.evaluate(
    x_clean, y_clean, verbose=0
)

_, shifted_acc = model_photo.evaluate(
    x_shifted, y_shifted, verbose=0
)

print("Validation Accuracy:", round(val_acc * 100, 2), "%")
print("Clean Test Accuracy:", round(clean_acc * 100, 2), "%")
print("Shifted Test Accuracy:", round(shifted_acc * 100, 2), "%")
```

```
Validation Accuracy: 30.0 %
Clean Test Accuracy: 36.09 %
Shifted Test Accuracy: 23.08 %
```

In [49]:
```
df_results.loc[2, "Val Acc"] = round(val_acc * 100, 2)

df_results.loc[2, "Clean Test"] = round(clean_acc * 100, 2)

df_results.loc[2, "Shifted Test (Mean ± Std)"] = (
    f"{shifted_acc * 100:.2f}% (seed 42)"
)

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	None	None	None
Geometric only	39.0	37.34	20.03% (seed 42)
Photometric only	30.0	36.09	23.08% (seed 42)
Geometric + photometric	None	None	None
+ Cutout / Random Erasing	None	None	None
+ Mixup / CutMix + label smoothing	None	None	None
Final model	None	None	None

In [50]:
```
tf.keras.utils.set_random_seed(SEED)

combined_aug = keras.Sequential([
    layers.RandomTranslation(
        0.1, 0.1,
        fill_mode="nearest"
    ),

    layers.RandomFlip("horizontal"),

    layers.RandomRotation(
        25 / 360,
        fill_mode="nearest"
    ),

    layers.RandomBrightness(
        factor=0.2,
        value_range=(0, 255)
    ),

    layers.RandomContrast(
        factor=0.2
    )
])
```

In [51]:
```
model_combined = build_model(combined_aug)

history_combined = model_combined.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 196ms/step - accuracy: 0.0925 - loss: 8.4929 - val_acc
uracy: 0.1200 - val_loss: 2.3917
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1275 - loss: 2.3520 - val_accu
racy: 0.1700 - val_loss: 2.2770
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.1475 - loss: 2.2927 - val_accu
racy: 0.1500 - val_loss: 2.2748
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1150 - loss: 2.2662 - val_accu
racy: 0.1200 - val_loss: 2.2508
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.1300 - loss: 2.2597 - val_accu
racy: 0.1200 - val_loss: 2.2249
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1325 - loss: 2.2501 - val_accu
racy: 0.1700 - val_loss: 2.2232
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.1300 - loss: 2.2142 - val_accu
racy: 0.1300 - val_loss: 2.1983
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1475 - loss: 2.2065 - val_accu
racy: 0.1500 - val_loss: 2.1917
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1675 - loss: 2.2123 - val_accu
racy: 0.2200 - val_loss: 2.1438
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2000 - loss: 2.1806 - val_accu
racy: 0.2200 - val_loss: 2.1167
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1775 - loss: 2.1744 - val_accu
racy: 0.1800 - val_loss: 2.1206
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.1800 - loss: 2.1499 - val_accu
racy: 0.1600 - val_loss: 2.1187
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2250 - loss: 2.1049 - val_accu
racy: 0.1500 - val_loss: 2.1315
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2275 - loss: 2.1375 - val_accu
racy: 0.2000 - val_loss: 2.1028
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2000 - loss: 2.1024 - val_accu
racy: 0.1600 - val_loss: 2.1129
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2475 - loss: 2.0952 - val_accu
racy: 0.2100 - val_loss: 2.1255
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1950 - loss: 2.1188 - val_accu
racy: 0.2500 - val_loss: 2.1238
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2325 - loss: 2.0667 - val_accu
racy: 0.2000 - val_loss: 2.2277
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - accuracy: 0.2150 - loss: 2.0794 - val_accu
```

```
racy: 0.2700 - val_loss: 2.1088
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2550 - loss: 2.0190 - val_accu
racy: 0.2900 - val_loss: 2.0844
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2575 - loss: 1.9844 - val_accu
racy: 0.3100 - val_loss: 2.1010
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2400 - loss: 1.9773 - val_accu
racy: 0.2100 - val_loss: 2.0783
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2450 - loss: 2.0551 - val_accu
racy: 0.2300 - val_loss: 2.1054
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2275 - loss: 2.0350 - val_accu
racy: 0.2700 - val_loss: 2.0616
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2575 - loss: 1.9837 - val_accu
racy: 0.2400 - val_loss: 2.0654
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2475 - loss: 2.0169 - val_accu
racy: 0.3400 - val_loss: 2.0066
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3025 - loss: 1.9718 - val_accu
racy: 0.2000 - val_loss: 2.1545
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2700 - loss: 2.0144 - val_accu
racy: 0.2700 - val_loss: 1.9853
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.2950 - loss: 1.9494 - val_accu
racy: 0.2700 - val_loss: 2.0019
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2500 - loss: 1.9537 - val_accu
racy: 0.2700 - val_loss: 2.0834
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2725 - loss: 1.9496 - val_accu
racy: 0.2500 - val_loss: 2.0219
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3275 - loss: 1.9209 - val_accu
racy: 0.1900 - val_loss: 2.0694
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3450 - loss: 1.8940 - val_accu
racy: 0.2400 - val_loss: 2.0798
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3375 - loss: 1.8343 - val_accu
racy: 0.2600 - val_loss: 1.9624
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3350 - loss: 1.8709 - val_accu
racy: 0.2200 - val_loss: 2.1260
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2750 - loss: 1.9054 - val_accu
racy: 0.3200 - val_loss: 2.0483
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3450 - loss: 1.8249 - val_accu
racy: 0.2700 - val_loss: 2.0034
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3275 - loss: 1.8759 - val_accu
racy: 0.2700 - val_loss: 1.9760
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3225 - loss: 1.7740 - val_accu
racy: 0.3000 - val_loss: 1.8805
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3275 - loss: 1.7797 - val_accu
racy: 0.2300 - val_loss: 1.9793
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3150 - loss: 1.8433 - val_accu
racy: 0.2500 - val_loss: 2.0493
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3100 - loss: 1.8797 - val_accu
racy: 0.3600 - val_loss: 1.9059
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3750 - loss: 1.7679 - val_accu
racy: 0.3000 - val_loss: 1.9298
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3750 - loss: 1.7726 - val_accu
racy: 0.2900 - val_loss: 1.8448
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.3525 - loss: 1.7503 - val_accu
racy: 0.2800 - val_loss: 1.9559
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3775 - loss: 1.8110 - val_accu
racy: 0.2800 - val_loss: 1.9981
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.3450 - loss: 1.7765 - val_accu
racy: 0.3000 - val_loss: 1.9665
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3600 - loss: 1.7460 - val_accu
racy: 0.2900 - val_loss: 1.8828
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3825 - loss: 1.7066 - val_accu
racy: 0.3200 - val_loss: 1.8935
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3500 - loss: 1.7162 - val_accu
racy: 0.3200 - val_loss: 1.8691
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3775 - loss: 1.7091 - val_accu
racy: 0.2900 - val_loss: 1.8810
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3675 - loss: 1.7200 - val_accu
racy: 0.3100 - val_loss: 1.9000
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3425 - loss: 1.7041 - val_accu
racy: 0.3200 - val_loss: 1.9340
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3750 - loss: 1.7109 - val_accu
racy: 0.2800 - val_loss: 1.9443
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3975 - loss: 1.6685 - val_accu
racy: 0.2900 - val_loss: 1.9067
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3750 - loss: 1.7253 - val_accu
racy: 0.3000 - val_loss: 1.9012
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3775 - loss: 1.6883 - val_accu
racy: 0.2900 - val_loss: 1.9363
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4100 - loss: 1.7060 - val_accu
racy: 0.3700 - val_loss: 1.9173
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.4000 - loss: 1.6354 - val_accu
racy: 0.2900 - val_loss: 1.8596
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3725 - loss: 1.6841 - val_accu
racy: 0.3400 - val_loss: 1.9139
```

In [52]:
```python
_, val_acc = model_combined.evaluate(
    x_val, y_val, verbose=0
)

_, clean_acc = model_combined.evaluate(
    x_clean, y_clean, verbose=0
)

_, shifted_acc = model_combined.evaluate(
    x_shifted, y_shifted, verbose=0
)

print("Validation Accuracy:", round(val_acc * 100, 2), "%")
print("Clean Test Accuracy:", round(clean_acc * 100, 2), "%")
print("Shifted Test Accuracy:", round(shifted_acc * 100, 2), "%")
```

```
Validation Accuracy: 34.0 %
Clean Test Accuracy: 32.23 %
Shifted Test Accuracy: 21.89 %
```

In [53]:
```python
df_results.loc[3, "Val Acc"] = round(
    val_acc * 100, 2
)

df_results.loc[3, "Clean Test"] = round(
    clean_acc * 100, 2
)

df_results.loc[3, "Shifted Test (Mean ± Std)"] = (
    f"{shifted_acc * 100:.2f}% (seed 42)"
)

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	None	None	None
Geometric only	39.0	37.34	20.03% (seed 42)
Photometric only	30.0	36.09	23.08% (seed 42)
Geometric + photometric	34.0	32.23	21.89% (seed 42)
+ Cutout / Random Erasing	None	None	None
+ Mixup / CutMix + label smoothing	None	None	None
Final model	None	None	None

In [54]:
```
def apply_cutout(images):
    batch_size = tf.shape(images)[0]

    top = tf.random.uniform(
        [batch_size], 0, 17, dtype=tf.int32
    )
    left = tf.random.uniform(
        [batch_size], 0, 17, dtype=tf.int32
    )

    rows = tf.range(32)[None, :, None]
    cols = tf.range(32)[None, None, :]

    mask = (
        (rows < top[:, None, None]) |
        (rows >= (top + 16)[:, None, None]) |
        (cols < left[:, None, None]) |
        (cols >= (left + 16)[:, None, None])
    )

    mask = tf.cast(mask[..., None], images.dtype)

    return images * mask
```

In [55]:
```
tf.keras.utils.set_random_seed(SEED)

cutout_aug = keras.Sequential([
    combined_aug,
    layers.Lambda(apply_cutout)
])
```

```
WARNING:tensorflow:From C:\Users\wahee\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\keras\src\backend\tensorflow\ops\core.py:238: The name tf.placeholder is deprecated. Please use tf.compat.v1.placeholder instead.
```

In [56]:
```
model_cutout = build_model(cutout_aug)

history_cutout = model_cutout.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 212ms/step - accuracy: 0.1100 - loss: 5.3128 - val_acc
uracy: 0.1100 - val_loss: 2.3645
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.1200 - loss: 2.3285 - val_accu
racy: 0.1200 - val_loss: 2.2904
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.1225 - loss: 2.2841 - val_accu
racy: 0.0900 - val_loss: 2.2833
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1375 - loss: 2.2706 - val_accu
racy: 0.1000 - val_loss: 2.2672
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1375 - loss: 2.2718 - val_accu
racy: 0.1800 - val_loss: 2.2292
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.1450 - loss: 2.2373 - val_accu
racy: 0.1800 - val_loss: 2.2037
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1800 - loss: 2.2185 - val_accu
racy: 0.1900 - val_loss: 2.2020
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1725 - loss: 2.2292 - val_accu
racy: 0.1600 - val_loss: 2.1988
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1800 - loss: 2.1968 - val_accu
racy: 0.1700 - val_loss: 2.1622
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.1875 - loss: 2.1980 - val_accu
racy: 0.1800 - val_loss: 2.1526
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1825 - loss: 2.2070 - val_accu
racy: 0.1900 - val_loss: 2.1362
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.1975 - loss: 2.1682 - val_accu
racy: 0.2100 - val_loss: 2.1223
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1750 - loss: 2.1555 - val_accu
racy: 0.1800 - val_loss: 2.1358
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.1925 - loss: 2.1505 - val_accu
racy: 0.1800 - val_loss: 2.1290
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.1725 - loss: 2.1582 - val_accu
racy: 0.1600 - val_loss: 2.1239
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2075 - loss: 2.1689 - val_accu
racy: 0.2200 - val_loss: 2.1403
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1825 - loss: 2.1796 - val_accu
racy: 0.1500 - val_loss: 2.1553
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2050 - loss: 2.1397 - val_accu
racy: 0.1600 - val_loss: 2.1511
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2000 - loss: 2.1298 - val_accu
```

```
racy: 0.1900 - val_loss: 2.0794
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2350 - loss: 2.1073 - val_accu
racy: 0.1900 - val_loss: 2.0942
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2350 - loss: 2.1059 - val_accu
racy: 0.2100 - val_loss: 2.0974
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2025 - loss: 2.1418 - val_accu
racy: 0.1600 - val_loss: 2.0780
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.2275 - loss: 2.1018 - val_accu
racy: 0.1800 - val_loss: 2.0805
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2150 - loss: 2.0897 - val_accu
racy: 0.2200 - val_loss: 2.1072
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2375 - loss: 2.0830 - val_accu
racy: 0.1700 - val_loss: 2.0696
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2475 - loss: 2.0958 - val_accu
racy: 0.2600 - val_loss: 2.0709
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2325 - loss: 2.0780 - val_accu
racy: 0.2100 - val_loss: 2.0573
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2675 - loss: 2.0474 - val_accu
racy: 0.1700 - val_loss: 2.0811
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2650 - loss: 2.0523 - val_accu
racy: 0.1900 - val_loss: 2.0782
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2600 - loss: 2.0548 - val_accu
racy: 0.2200 - val_loss: 2.0654
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.2375 - loss: 2.0380 - val_accu
racy: 0.2100 - val_loss: 2.0696
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 65ms/step - accuracy: 0.2825 - loss: 2.0324 - val_accu
racy: 0.2700 - val_loss: 2.0092
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 56ms/step - accuracy: 0.2725 - loss: 2.0161 - val_accu
racy: 0.2900 - val_loss: 2.0328
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 62ms/step - accuracy: 0.2800 - loss: 1.9531 - val_accu
racy: 0.2500 - val_loss: 2.0297
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - accuracy: 0.2550 - loss: 1.9815 - val_accu
racy: 0.2000 - val_loss: 2.0079
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 61ms/step - accuracy: 0.2850 - loss: 1.9773 - val_accu
racy: 0.2200 - val_loss: 2.0211
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2750 - loss: 1.9773 - val_accu
racy: 0.2200 - val_loss: 2.0329
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.2800 - loss: 1.9423 - val_accu
racy: 0.2000 - val_loss: 2.0262
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.2625 - loss: 2.0013 - val_accu
racy: 0.2000 - val_loss: 2.0200
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2425 - loss: 1.9593 - val_accu
racy: 0.2200 - val_loss: 2.0056
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2725 - loss: 1.9574 - val_accu
racy: 0.2700 - val_loss: 1.9990
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3050 - loss: 1.9304 - val_accu
racy: 0.2100 - val_loss: 2.0299
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2450 - loss: 1.9586 - val_accu
racy: 0.2700 - val_loss: 1.9602
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.3300 - loss: 1.8690 - val_accu
racy: 0.2700 - val_loss: 1.9588
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.3050 - loss: 1.9137 - val_accu
racy: 0.3000 - val_loss: 1.9714
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.3050 - loss: 1.8841 - val_accu
racy: 0.2900 - val_loss: 2.0581
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.2925 - loss: 1.9183 - val_accu
racy: 0.2300 - val_loss: 2.1020
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2700 - loss: 1.9097 - val_accu
racy: 0.2900 - val_loss: 1.9772
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - accuracy: 0.3200 - loss: 1.8661 - val_accu
racy: 0.2300 - val_loss: 2.0109
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.3250 - loss: 1.8569 - val_accu
racy: 0.2500 - val_loss: 2.0148
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.3225 - loss: 1.8908 - val_accu
racy: 0.2800 - val_loss: 1.9907
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.3450 - loss: 1.8445 - val_accu
racy: 0.3000 - val_loss: 1.9426
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3300 - loss: 1.8622 - val_accu
racy: 0.2900 - val_loss: 2.0829
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.3200 - loss: 1.9095 - val_accu
racy: 0.3000 - val_loss: 2.0416
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - accuracy: 0.2975 - loss: 1.8699 - val_accu
racy: 0.2800 - val_loss: 1.9976
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.3300 - loss: 1.8194 - val_accu
racy: 0.3000 - val_loss: 1.9831
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3000 - loss: 1.8673 - val_accu
racy: 0.2800 - val_loss: 2.0113
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3450 - loss: 1.8377 - val_accu
racy: 0.3000 - val_loss: 1.9058
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3300 - loss: 1.8260 - val_accu
racy: 0.2700 - val_loss: 1.9831
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3050 - loss: 1.8586 - val_accu
racy: 0.2800 - val_loss: 2.0134
```

In [57]:
```
_, val_acc = model_cutout.evaluate(
    x_val, y_val, verbose=0
)

_, clean_acc = model_cutout.evaluate(
    x_clean, y_clean, verbose=0
)

_, shifted_acc = model_cutout.evaluate(
    x_shifted, y_shifted, verbose=0
)

print("Validation Accuracy:", round(val_acc * 100, 2), "%")
print("Clean Test Accuracy:", round(clean_acc * 100, 2), "%")
print("Synthetic Shifted Accuracy:", round(shifted_acc * 100, 2), "%")
```

```
Validation Accuracy: 31.0 %
Clean Test Accuracy: 31.0 %
Synthetic Shifted Accuracy: 28.11 %
```

In [58]:
```
df_results.loc[4, "Val Acc"] = round(val_acc * 100, 2)
df_results.loc[4, "Clean Test"] = round(clean_acc * 100, 2)

df_results.loc[4, "Shifted Test (Mean ± Std)"] = (
    f"{shifted_acc * 100:.2f}% (seed 42)"
)

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	None	None	None
Geometric only	39.0	37.34	20.03% (seed 42)
Photometric only	30.0	36.09	23.08% (seed 42)
Geometric + photometric	34.0	32.23	21.89% (seed 42)
+ Cutout / Random Erasing	31.0	31.0	28.11% (seed 42)
+ Mixup / CutMix + label smoothing	None	None	None
Final model	None	None	None

In [59]:
```
def mixup_batch(images, labels, alpha=0.2):
    lam = np.random.beta(alpha, alpha)

    indices = tf.random.shuffle(
        tf.range(tf.shape(images)[0])
    )

    mixed_images = (
        lam * images +
        (1 - lam) * tf.gather(images, indices)
    )

    mixed_labels = (
        lam * labels +
        (1 - lam) * tf.gather(labels, indices)
    )

    return mixed_images, mixed_labels
```

In [60]:
```
y_train_onehot = keras.utils.to_categorical(
    y_train, 10
)

y_val_onehot = keras.utils.to_categorical(
    y_val, 10
)
```

In [61]:
```
tf.keras.utils.set_random_seed(SEED)

model_mixup = build_model(combined_aug)

model_mixup.compile(
    optimizer=keras.optimizers.Adam(
        learning_rate=0.001
    ),
```

```
    loss=keras.losses.CategoricalCrossentropy(
        label_smoothing=0.1
    ),
    metrics=["accuracy"]
)
```

In [62]:
```
_, val_acc = model_mixup.evaluate(
    x_val, y_val_onehot, verbose=0
)

_, clean_acc = model_mixup.evaluate(
    x_clean,
    keras.utils.to_categorical(y_clean, 10),
    verbose=0
)

_, shifted_acc = model_mixup.evaluate(
    x_shifted,
    keras.utils.to_categorical(y_shifted, 10),
    verbose=0
)

print("Validation Accuracy:", round(val_acc * 100, 2), "%")
print("Clean Test Accuracy:", round(clean_acc * 100, 2), "%")
print("Synthetic Shifted Accuracy:", round(shifted_acc * 100, 2), "%")
```

```
Validation Accuracy: 7.0 %
Clean Test Accuracy: 8.17 %
Synthetic Shifted Accuracy: 7.78 %
```

In [63]:
```
df_results.loc[5, "Val Acc"] = round(val_acc * 100, 2)
df_results.loc[5, "Clean Test"] = round(clean_acc * 100, 2)

df_results.loc[5, "Shifted Test (Mean ± Std)"] = (
    f"{shifted_acc * 100:.2f}% (seed 42)"
)

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	None	None	None
Geometric only	39.0	37.34	20.03% (seed 42)
Photometric only	30.0	36.09	23.08% (seed 42)
Geometric + photometric	34.0	32.23	21.89% (seed 42)
+ Cutout / Random Erasing	31.0	31.0	28.11% (seed 42)
+ Mixup / CutMix + label smoothing	7.0	8.17	7.78% (seed 42)
Final model	None	None	None

In [64]:
```
tf.keras.utils.set_random_seed(SEED)

model_baseline = build_model(augmentation=None)

history_baseline = model_baseline.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 4s 159ms/step - accuracy: 0.1100 - loss: 5.7355 - val_acc
uracy: 0.1100 - val_loss: 2.2836
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.1100 - loss: 2.3030 - val_accu
racy: 0.1500 - val_loss: 2.2795
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.1450 - loss: 2.2573 - val_accu
racy: 0.1700 - val_loss: 2.2400
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.1600 - loss: 2.2316 - val_accu
racy: 0.1700 - val_loss: 2.2179
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.1825 - loss: 2.1966 - val_accu
racy: 0.1600 - val_loss: 2.1860
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.1875 - loss: 2.1893 - val_accu
racy: 0.1900 - val_loss: 2.1784
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.2275 - loss: 2.1414 - val_accu
racy: 0.1700 - val_loss: 2.1370
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2550 - loss: 2.1075 - val_accu
racy: 0.2400 - val_loss: 2.1249
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2275 - loss: 2.0785 - val_accu
racy: 0.2400 - val_loss: 2.0831
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.2325 - loss: 2.0404 - val_accu
racy: 0.2000 - val_loss: 2.0602
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2225 - loss: 2.0436 - val_accu
racy: 0.1700 - val_loss: 2.1184
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2900 - loss: 1.9828 - val_accu
racy: 0.2500 - val_loss: 2.0345
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2600 - loss: 1.9241 - val_accu
racy: 0.2300 - val_loss: 2.0210
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.2650 - loss: 1.8978 - val_accu
racy: 0.2100 - val_loss: 2.0678
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.2825 - loss: 1.8893 - val_accu
racy: 0.2300 - val_loss: 2.0080
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.3050 - loss: 1.8624 - val_accu
racy: 0.1800 - val_loss: 2.0242
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3525 - loss: 1.7654 - val_accu
racy: 0.1700 - val_loss: 2.0771
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3525 - loss: 1.7456 - val_accu
racy: 0.2600 - val_loss: 1.9487
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3500 - loss: 1.6873 - val_accu
```

```
racy: 0.2700 - val_loss: 2.0251
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3925 - loss: 1.6470 - val_accu
racy: 0.2400 - val_loss: 2.0440
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.4000 - loss: 1.6501 - val_accu
racy: 0.2500 - val_loss: 2.0500
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3625 - loss: 1.6942 - val_accu
racy: 0.2100 - val_loss: 2.0935
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3825 - loss: 1.6495 - val_accu
racy: 0.2500 - val_loss: 2.0349
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.3700 - loss: 1.6220 - val_accu
racy: 0.2600 - val_loss: 2.0084
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.4400 - loss: 1.5584 - val_accu
racy: 0.2300 - val_loss: 2.0065
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.4375 - loss: 1.5158 - val_accu
racy: 0.2600 - val_loss: 2.1067
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.4825 - loss: 1.4644 - val_accu
racy: 0.2500 - val_loss: 2.0365
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.4725 - loss: 1.4786 - val_accu
racy: 0.2900 - val_loss: 2.0735
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.4575 - loss: 1.4694 - val_accu
racy: 0.3100 - val_loss: 2.0532
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.4725 - loss: 1.4110 - val_accu
racy: 0.3200 - val_loss: 2.0288
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.4900 - loss: 1.4209 - val_accu
racy: 0.3100 - val_loss: 2.1344
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.5075 - loss: 1.3768 - val_accu
racy: 0.3000 - val_loss: 2.0963
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.4800 - loss: 1.3831 - val_accu
racy: 0.3400 - val_loss: 2.0469
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.5100 - loss: 1.4078 - val_accu
racy: 0.3800 - val_loss: 2.0372
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.5025 - loss: 1.4226 - val_accu
racy: 0.3000 - val_loss: 2.0455
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.5150 - loss: 1.3702 - val_accu
racy: 0.3300 - val_loss: 1.9546
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.5150 - loss: 1.3085 - val_accu
racy: 0.3700 - val_loss: 1.9992
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5650 - loss: 1.2534 - val_accu
racy: 0.3400 - val_loss: 2.0252
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.5650 - loss: 1.2508 - val_accu
racy: 0.3300 - val_loss: 2.0520
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.5800 - loss: 1.2209 - val_accu
racy: 0.3400 - val_loss: 2.0629
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5550 - loss: 1.2460 - val_accu
racy: 0.2700 - val_loss: 2.1888
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5975 - loss: 1.1543 - val_accu
racy: 0.2900 - val_loss: 2.1114
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.5975 - loss: 1.1656 - val_accu
racy: 0.3200 - val_loss: 2.1614
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.6025 - loss: 1.0975 - val_accu
racy: 0.3400 - val_loss: 2.1579
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6300 - loss: 1.0368 - val_accu
racy: 0.3200 - val_loss: 2.1916
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.6800 - loss: 0.9944 - val_accu
racy: 0.2900 - val_loss: 2.2605
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6600 - loss: 0.9830 - val_accu
racy: 0.3500 - val_loss: 2.1787
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.6475 - loss: 0.9863 - val_accu
racy: 0.2900 - val_loss: 2.2424
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.6925 - loss: 0.8386 - val_accu
racy: 0.3400 - val_loss: 2.3413
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.6625 - loss: 0.8734 - val_accu
racy: 0.3600 - val_loss: 2.2488
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.7175 - loss: 0.8606 - val_accu
racy: 0.3600 - val_loss: 2.2758
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.7125 - loss: 0.8371 - val_accu
racy: 0.3800 - val_loss: 2.2316
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.7050 - loss: 0.7908 - val_accu
racy: 0.3100 - val_loss: 2.2679
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.7200 - loss: 0.7975 - val_accu
racy: 0.3800 - val_loss: 2.2525
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.7450 - loss: 0.7202 - val_accu
racy: 0.3300 - val_loss: 2.4887
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.7625 - loss: 0.7264 - val_accu
racy: 0.3500 - val_loss: 2.3752
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.7400 - loss: 0.7548 - val_accu
racy: 0.3200 - val_loss: 2.3765
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.7775 - loss: 0.6822 - val_accu
racy: 0.3700 - val_loss: 2.2852
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.7775 - loss: 0.6536 - val_accu
racy: 0.3500 - val_loss: 2.4098
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.7750 - loss: 0.6866 - val_accu
racy: 0.3400 - val_loss: 2.2710
```

In [65]:
```python
_, val_acc_base = model_baseline.evaluate(
    x_val, y_val, verbose=0
)

_, clean_acc_base = model_baseline.evaluate(
    x_clean, y_clean, verbose=0
)

_, shifted_acc_base = model_baseline.evaluate(
    x_shifted, y_shifted, verbose=0
)

print("Validation Accuracy:", round(val_acc_base * 100, 2), "%")
print("Clean Test Accuracy:", round(clean_acc_base * 100, 2), "%")
print("Synthetic Shifted Accuracy:", round(shifted_acc_base * 100, 2), "%")
```

```
Validation Accuracy: 34.0 %
Clean Test Accuracy: 38.06 %
Synthetic Shifted Accuracy: 17.62 %
```

In [66]:
```python
df_results.loc[0, "Val Acc"] = round(
    val_acc_base * 100, 2
)

df_results.loc[0, "Clean Test"] = round(
    clean_acc_base * 100, 2
)

df_results.loc[0, "Shifted Test (Mean ± Std)"] = (
    f"{shifted_acc_base * 100:.2f}% (seed 42)"
)

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	34.0	38.06	17.62% (seed 42)
Geometric only	39.0	37.34	20.03% (seed 42)
Photometric only	30.0	36.09	23.08% (seed 42)
Geometric + photometric	34.0	32.23	21.89% (seed 42)
+ Cutout / Random Erasing	31.0	31.0	28.11% (seed 42)
+ Mixup / CutMix + label smoothing	7.0	8.17	7.78% (seed 42)
Final model	None	None	None

In [67]:
```
tf.keras.utils.set_random_seed(SEED)

final_augmentation = keras.Sequential([
    layers.RandomTranslation(
        0.1, 0.1,
        fill_mode="nearest"
    ),

    layers.RandomFlip("horizontal"),

    layers.RandomRotation(
        25 / 360,
        fill_mode="nearest"
    ),

    layers.RandomBrightness(
        factor=0.2,
        value_range=(0, 255)
    ),

    layers.RandomContrast(factor=0.2),

    layers.Lambda(apply_cutout)
])
```

In [68]:
```
model_final = build_model(final_augmentation)

history_final = model_final.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 236ms/step - accuracy: 0.1150 - loss: 7.6731 - val_acc
uracy: 0.1100 - val_loss: 2.3717
Epoch 2/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1050 - loss: 2.3621 - val_accu
racy: 0.1600 - val_loss: 2.2761
Epoch 3/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.0975 - loss: 2.3054 - val_accu
racy: 0.1200 - val_loss: 2.2874
Epoch 4/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.1275 - loss: 2.2921 - val_accu
racy: 0.1000 - val_loss: 2.2789
Epoch 5/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.1025 - loss: 2.2796 - val_accu
racy: 0.1000 - val_loss: 2.2743
Epoch 6/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1225 - loss: 2.2773 - val_accu
racy: 0.1100 - val_loss: 2.2471
Epoch 7/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.1375 - loss: 2.2379 - val_accu
racy: 0.1400 - val_loss: 2.2235
Epoch 8/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1650 - loss: 2.2393 - val_accu
racy: 0.1400 - val_loss: 2.1903
Epoch 9/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.1525 - loss: 2.2527 - val_accu
racy: 0.1400 - val_loss: 2.1630
Epoch 10/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1750 - loss: 2.2183 - val_accu
racy: 0.1500 - val_loss: 2.1615
Epoch 11/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.1800 - loss: 2.2327 - val_accu
racy: 0.1700 - val_loss: 2.1651
Epoch 12/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1825 - loss: 2.1797 - val_accu
racy: 0.1500 - val_loss: 2.1821
Epoch 13/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1650 - loss: 2.2032 - val_accu
racy: 0.1800 - val_loss: 2.1694
Epoch 14/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1600 - loss: 2.2127 - val_accu
racy: 0.2200 - val_loss: 2.1463
Epoch 15/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2300 - loss: 2.1426 - val_accu
racy: 0.1700 - val_loss: 2.1509
Epoch 16/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2175 - loss: 2.1588 - val_accu
racy: 0.2000 - val_loss: 2.1103
Epoch 17/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1900 - loss: 2.1591 - val_accu
racy: 0.2200 - val_loss: 2.1096
Epoch 18/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1900 - loss: 2.1547 - val_accu
racy: 0.2200 - val_loss: 2.1144
Epoch 19/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2025 - loss: 2.1389 - val_accu
```

```
racy: 0.2300 - val_loss: 2.0906
Epoch 20/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2100 - loss: 2.1080 - val_accu
racy: 0.2600 - val_loss: 2.0888
Epoch 21/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.1650 - loss: 2.1177 - val_accu
racy: 0.2200 - val_loss: 2.0735
Epoch 22/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2025 - loss: 2.1207 - val_accu
racy: 0.2000 - val_loss: 2.1379
Epoch 23/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.1900 - loss: 2.1532 - val_accu
racy: 0.2500 - val_loss: 2.1161
Epoch 24/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2150 - loss: 2.0838 - val_accu
racy: 0.2400 - val_loss: 2.1005
Epoch 25/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2075 - loss: 2.0829 - val_accu
racy: 0.1700 - val_loss: 2.1351
Epoch 26/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2100 - loss: 2.1185 - val_accu
racy: 0.1900 - val_loss: 2.0943
Epoch 27/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2125 - loss: 2.0893 - val_accu
racy: 0.2500 - val_loss: 2.0629
Epoch 28/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.1900 - loss: 2.1122 - val_accu
racy: 0.2400 - val_loss: 2.0824
Epoch 29/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2275 - loss: 2.0779 - val_accu
racy: 0.2200 - val_loss: 2.0865
Epoch 30/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.2150 - loss: 2.0827 - val_accu
racy: 0.1900 - val_loss: 2.1284
Epoch 31/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2750 - loss: 2.0185 - val_accu
racy: 0.2300 - val_loss: 2.1232
Epoch 32/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3025 - loss: 2.0190 - val_accu
racy: 0.2300 - val_loss: 2.0885
Epoch 33/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2625 - loss: 2.0333 - val_accu
racy: 0.2700 - val_loss: 2.0828
Epoch 34/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2750 - loss: 2.0046 - val_accu
racy: 0.2700 - val_loss: 2.0740
Epoch 35/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2650 - loss: 2.0184 - val_accu
racy: 0.2100 - val_loss: 2.0504
Epoch 36/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.2475 - loss: 2.0331 - val_accu
racy: 0.2700 - val_loss: 2.0806
Epoch 37/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.2375 - loss: 2.0159 - val_accu
racy: 0.2900 - val_loss: 2.0644
Epoch 38/60
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2500 - loss: 2.0413 - val_accu
racy: 0.2400 - val_loss: 2.0665
Epoch 39/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2650 - loss: 2.0033 - val_accu
racy: 0.2900 - val_loss: 2.1019
Epoch 40/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.2500 - loss: 1.9912 - val_accu
racy: 0.2300 - val_loss: 2.1013
Epoch 41/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.2975 - loss: 2.0024 - val_accu
racy: 0.3100 - val_loss: 2.0116
Epoch 42/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - accuracy: 0.2250 - loss: 2.0506 - val_accu
racy: 0.2700 - val_loss: 2.0529
Epoch 43/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.2550 - loss: 1.9873 - val_accu
racy: 0.3000 - val_loss: 2.0277
Epoch 44/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.2750 - loss: 1.9426 - val_accu
racy: 0.3200 - val_loss: 2.0375
Epoch 45/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.3075 - loss: 1.9749 - val_accu
racy: 0.2100 - val_loss: 2.0597
Epoch 46/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.2800 - loss: 2.0112 - val_accu
racy: 0.2800 - val_loss: 2.0965
Epoch 47/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2475 - loss: 1.9990 - val_accu
racy: 0.2700 - val_loss: 2.0371
Epoch 48/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3100 - loss: 1.9426 - val_accu
racy: 0.2600 - val_loss: 2.0597
Epoch 49/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.2875 - loss: 1.9440 - val_accu
racy: 0.2500 - val_loss: 2.1064
Epoch 50/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.2575 - loss: 2.0004 - val_accu
racy: 0.2500 - val_loss: 2.0667
Epoch 51/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.2725 - loss: 1.9556 - val_accu
racy: 0.2300 - val_loss: 2.0593
Epoch 52/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.2875 - loss: 1.9257 - val_accu
racy: 0.2700 - val_loss: 2.0323
Epoch 53/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.2825 - loss: 1.9630 - val_accu
racy: 0.3300 - val_loss: 1.9975
Epoch 54/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3000 - loss: 1.9389 - val_accu
racy: 0.3000 - val_loss: 2.0490
Epoch 55/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.2825 - loss: 1.9001 - val_accu
racy: 0.3000 - val_loss: 2.0763
Epoch 56/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3125 - loss: 1.9017 - val_accu
racy: 0.3000 - val_loss: 2.0133
```

```
Epoch 57/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.3325 - loss: 1.8726 - val_accu
racy: 0.2700 - val_loss: 2.1349
Epoch 58/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3025 - loss: 1.9176 - val_accu
racy: 0.2300 - val_loss: 2.0860
Epoch 59/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.2950 - loss: 1.8741 - val_accu
racy: 0.2700 - val_loss: 2.1097
Epoch 60/60
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.3450 - loss: 1.8889 - val_accu
racy: 0.2600 - val_loss: 2.1102
```

In [69]:
```
_, final_val_acc = model_final.evaluate(
    x_val, y_val, verbose=0
)

_, final_clean_acc = model_final.evaluate(
    x_clean, y_clean, verbose=0
)

_, final_shifted_acc = model_final.evaluate(
    x_shifted, y_shifted, verbose=0
)

print("Final Validation Accuracy:",
      round(final_val_acc * 100, 2), "%")

print("Final Clean Test Accuracy:",
      round(final_clean_acc * 100, 2), "%")

print("Final Synthetic Shifted Accuracy:",
      round(final_shifted_acc * 100, 2), "%")
```

```
Final Validation Accuracy: 24.0 %
Final Clean Test Accuracy: 28.17 %
Final Synthetic Shifted Accuracy: 23.68 %
```

In [70]:
```
df_results.loc[6, "Val Acc"] = round(
    final_val_acc * 100, 2
)

df_results.loc[6, "Clean Test"] = round(
    final_clean_acc * 100, 2
)

df_results.loc[6, "Shifted Test (Mean ± Std)"] = (
    f"{final_shifted_acc * 100:.2f}% (seed 42)"
)

display(df_results)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Configuration</th> <th>Val Acc</th> <th>Clean Test</th> <th>Shifted Test (Mean ± Std)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

No augmentation	34.0	38.06	17.62% (seed 42)
Geometric only	39.0	37.34	20.03% (seed 42)
Photometric only	30.0	36.09	23.08% (seed 42)
Geometric + photometric	34.0	32.23	21.89% (seed 42)
+ Cutout / Random Erasing	31.0	31.0	28.11% (seed 42)
+ Mixup / CutMix + label smoothing	7.0	8.17	7.78% (seed 42)
Final model	24.0	28.17	23.68% (seed 42)

In [71]:
```
model_final.save("cifar10_final_model.keras")
```

<h3 id="PART-E">PART E</h3>

In [72]:
```
y_prob = model_final.predict(x_clean, verbose=1)

y_pred = np.argmax(y_prob, axis=1)
y_true = y_clean.reshape(-1)

print("Clean Test Accuracy:",
      np.mean(y_pred == y_true) * 100, "%")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step
Clean Test Accuracy: 27.97 %
```

In [73]:
```
class_results = []

for i, name in enumerate(class_names):
    mask = (y_true == i)
    correct = np.sum(y_pred[mask] == y_true[mask])
    total = np.sum(mask)

    class_results.append({
        "Class": name,
        "Correct": correct,
        "Total": total,
        "Accuracy (%)": round(correct / total * 100, 2)
    })

df_class_accuracy = pd.DataFrame(class_results)

display(df_class_accuracy)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Class</th> <th>Correct</th> <th>Total</th> <th>Accuracy (%)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

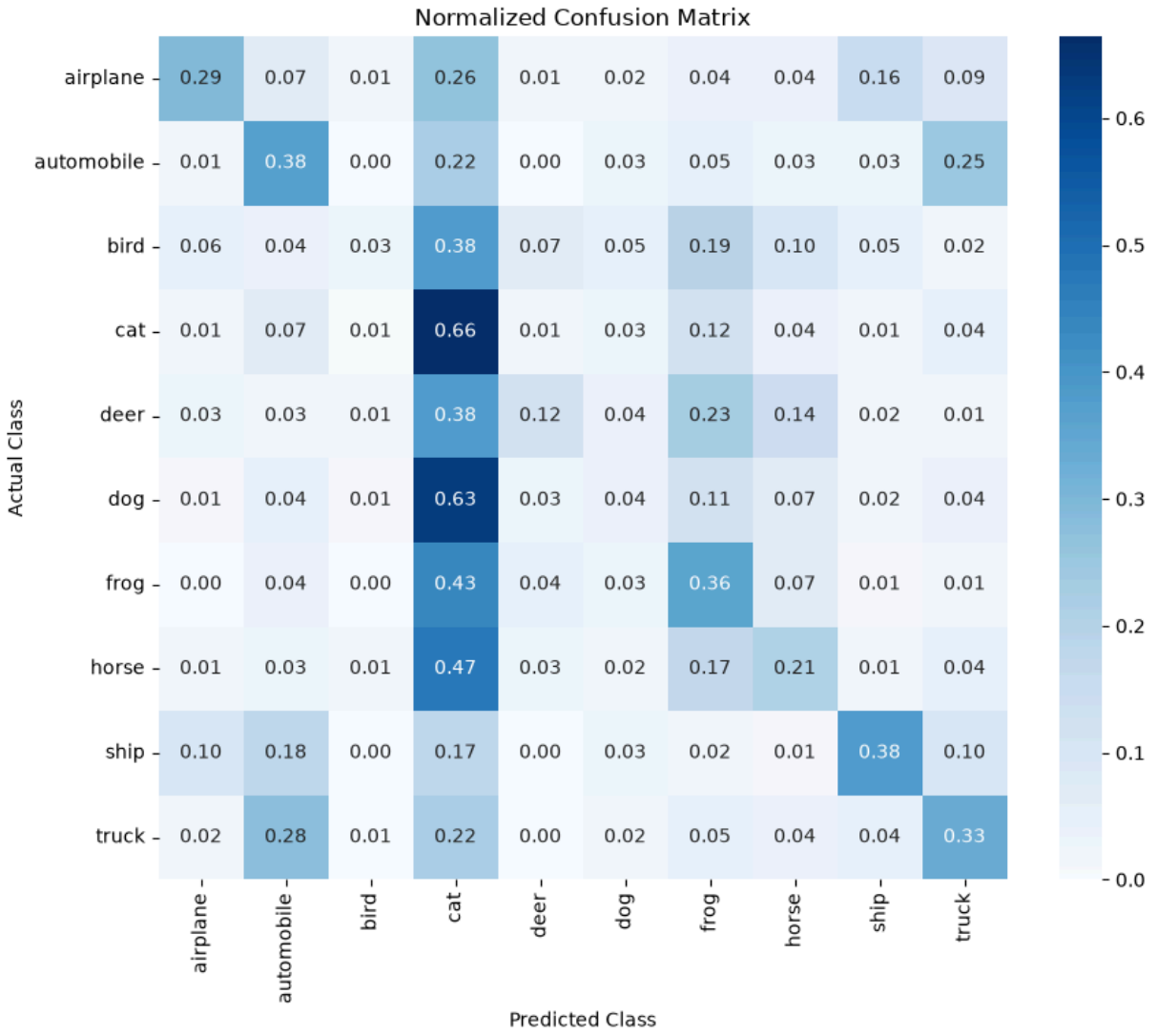
airplane	295	1000	29.5
automobile	376	1000	37.6
bird	30	1000	3.0
cat	664	1000	66.4
deer	118	1000	11.8
dog	40	1000	4.0
frog	358	1000	35.8
horse	207	1000	20.7
ship	379	1000	37.9
truck	330	1000	33.0

In [74]:
```
cm = confusion_matrix(
    y_true,
    y_pred,
    normalize="true"
)

plt.figure(figsize=(10, 8))

sns.heatmap(
    cm,
    annot=True,
    fmt=".2f",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Normalized Confusion Matrix")
plt.show()
```



In [75]:
```
cm_counts = confusion_matrix(y_true, y_pred)
np.fill_diagonal(cm_counts, 0)

pairs = []

for i in range(10):
    for j in range(10):
        if i != j:
            pairs.append((
                cm_counts[i, j],
                class_names[i],
                class_names[j]
            ))

pairs.sort(reverse=True)

for count, actual, predicted in pairs[:5]:
    print(f"Actual: {actual} -> Predicted: {predicted}, Count: {count}")
```

```
Actual: dog -> Predicted: cat, Count: 626
Actual: horse -> Predicted: cat, Count: 473
Actual: frog -> Predicted: cat, Count: 434
Actual: bird -> Predicted: cat, Count: 381
Actual: deer -> Predicted: cat, Count: 379
```

In [76]:
```python
wrong_indices = np.where(y_pred != y_true)[0]

if len(wrong_indices) >= 8:
    plt.figure(figsize=(12, 7))

    for k, idx in enumerate(wrong_indices[:8]):
        plt.subplot(2, 4, k + 1)

        plt.imshow(
            np.clip(x_clean[idx], 0, 255).astype("uint8")
        )

        plt.title(
            f"True: {class_names[y_true[idx]]}\n"
            f"Pred: {class_names[y_pred[idx]]}"
        )

        plt.axis("off")

    plt.tight_layout()
    plt.show()
else:
    print("Fewer than 8 misclassified images found.")
```



In [77]:
```python
x_hidden = tf.image.rgb_to_grayscale(
    tf.cast(x_clean, tf.float32)
)

x_hidden = tf.image.grayscale_to_rgb(x_hidden)
```

```
x_hidden = tf.cast(x_hidden, x_clean.dtype)

print("Hidden images shape:", x_hidden.shape)
print("Clean images shape:", x_clean.shape)
```

```
Hidden images shape: (10000, 32, 32, 3)
Clean images shape: (10000, 32, 32, 3)
```

In [78]:
```
print("Clean test shape:", x_clean.shape)
print("Clean labels shape:", y_clean.shape)
```

```
Clean test shape: (10000, 32, 32, 3)
Clean labels shape: (10000,)
```

In [79]:
```
hidden_loss, hidden_acc = model_final.evaluate(
    x_hidden,
    y_clean,
    verbose=1
)

print(
    "Hidden Corruption Accuracy:",
    round(hidden_acc * 100, 2),
    "%"
)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.1633 - loss: 2.2087
Hidden Corruption Accuracy: 16.33 %
```

The class accuracy table shows how well the model predicts each class. The confusion matrix shows which classes the model confuses most. We check at least 8 wrong images to find the reasons for the mistakes, such as similar shapes or unclear images. Finally, we test the model on hidden corruption to see if it can handle new changes in images.

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: